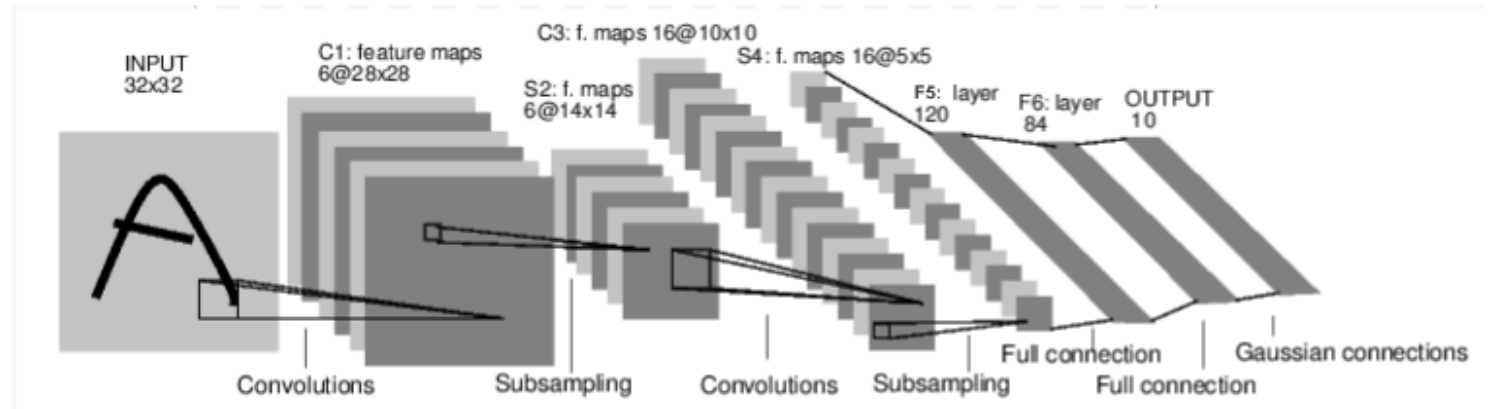


Deep Learning

LECTURE 3 - CONVOLUTIONAL NEURAL NETWORKS

Kevin Bryson



How many people recognize this image?
Have you seen it anywhere?

Overview

1. Last week we looked at MLPs (Multi-Layer Perceptrons; also called fully connected layers, dense linear layers, etc). The previous lab showed they were quite powerful for both classification and regression (for functions with small numbers of inputs).
2. I will explain why we do **not use them** for image classification (although the 3Blue1Brown video did !)
3. I will introduce Convolutional Neural Networks (CNNs or ConvNets) and how they work:
 1. CNN layers.
 2. Max pooling layers.
 3. Fully connected layers.
4. Followed by more technical detail about how CNN layers work (padding, stride, etc.)
5. Before covering some classic CNN Architectures like AlexNet, Google Inception, VGG16, etc.

Where have I seen this image ?



Run in Google Colab

Download Notebook

View on GitHub

Neural Networks

Define the network

Loss Function

Backprop

Update the weights

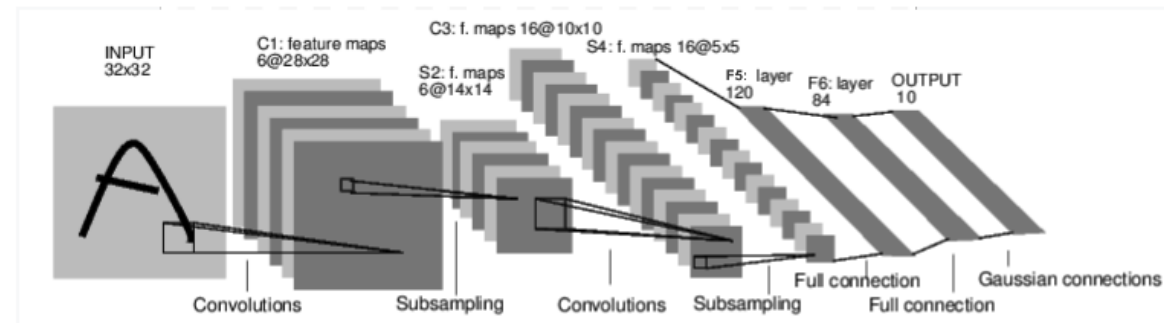
Neural Networks

Created On: Mar 24, 2017 | Last Updated: May 06, 2024 | Last Verified: Nov 05, 2024

Neural networks can be constructed using the `torch.nn` package.

Now that you had a glimpse of `autograd`, `nn` depends on `autograd` to define models and differentiate them. An `nn.Module` contains layers, and a method `forward(input)` that returns the `output`.

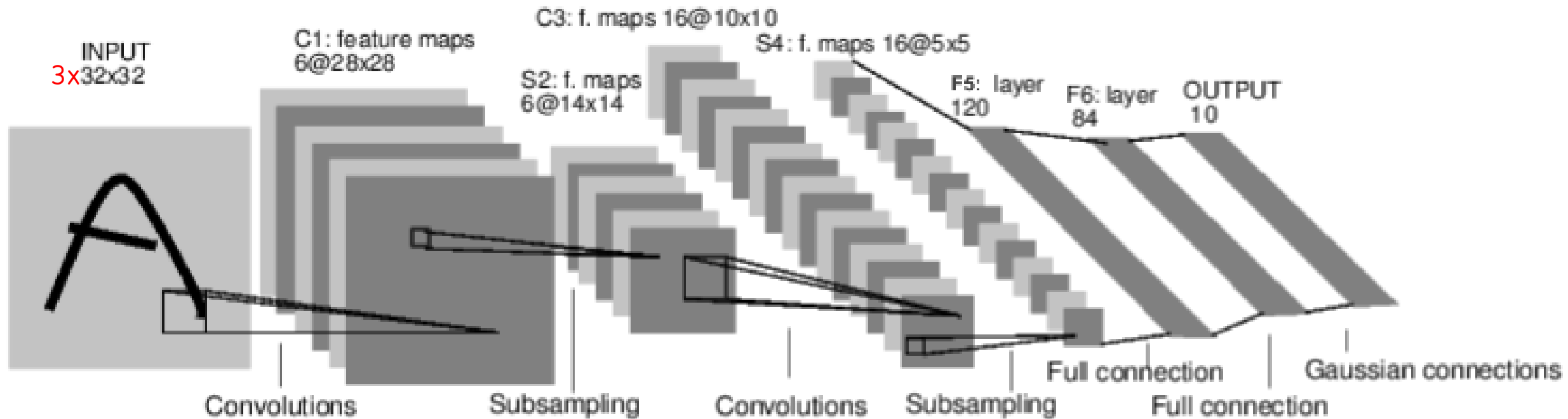
For example, look at this network that classifies digit images:



convnet

It is a simple feed-forward network. It takes the input, feeds it through several layers one after the other, and then finally gives the output.

I want to explain CNN concepts in terms of this classic model ... used in Lab 1 ...



I'm modified this image slightly. What type of input data is this using ?
How many outputs ? Have you seen an example of this type of data?

This is the architecture of the CNN model in Lab 1 for CIFAR10 classification

2. Define a Convolutional Neural Network

Copy the neural network from the Neural Networks section before and modify it to take 3-channel images (instead of 1-channel images as it was defined).

```
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
```

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



By the end of this lecture you should know what the terms mean here (and how they relate to PyTorch code)

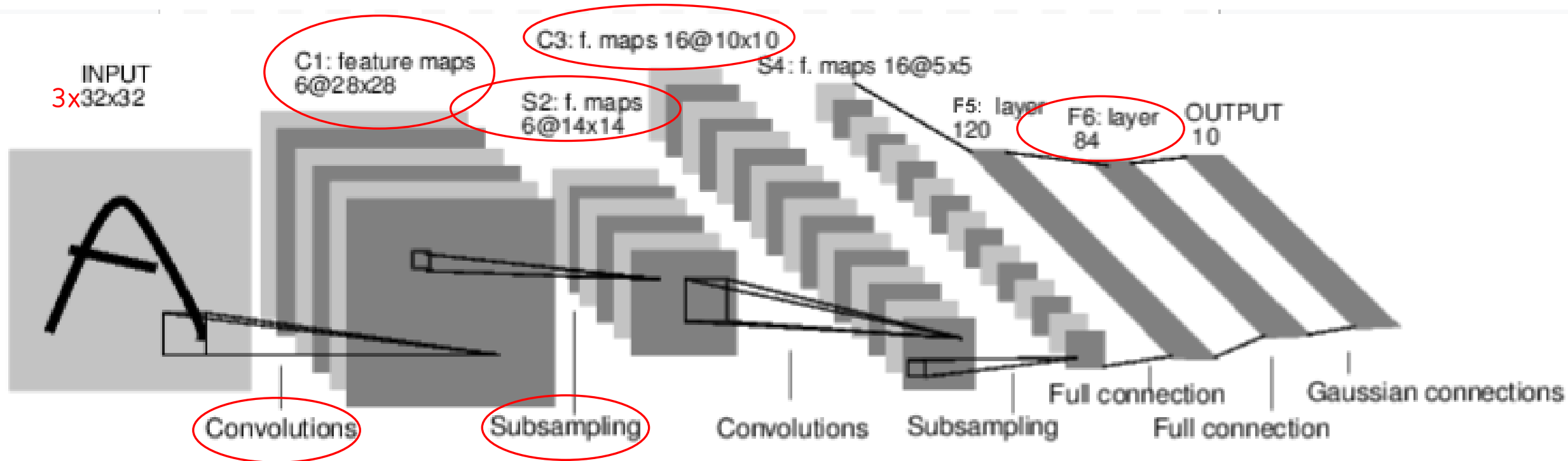
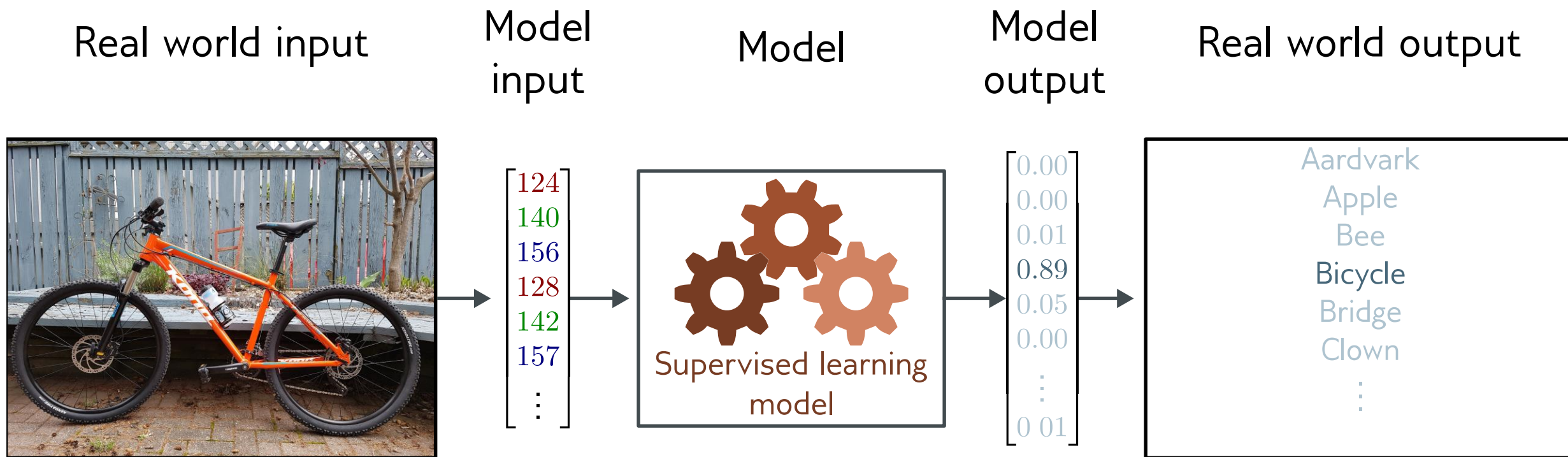


Image classification



- Multiclass classification problem (discrete classes, >2 possible classes)
- Convolutional network

Object detection

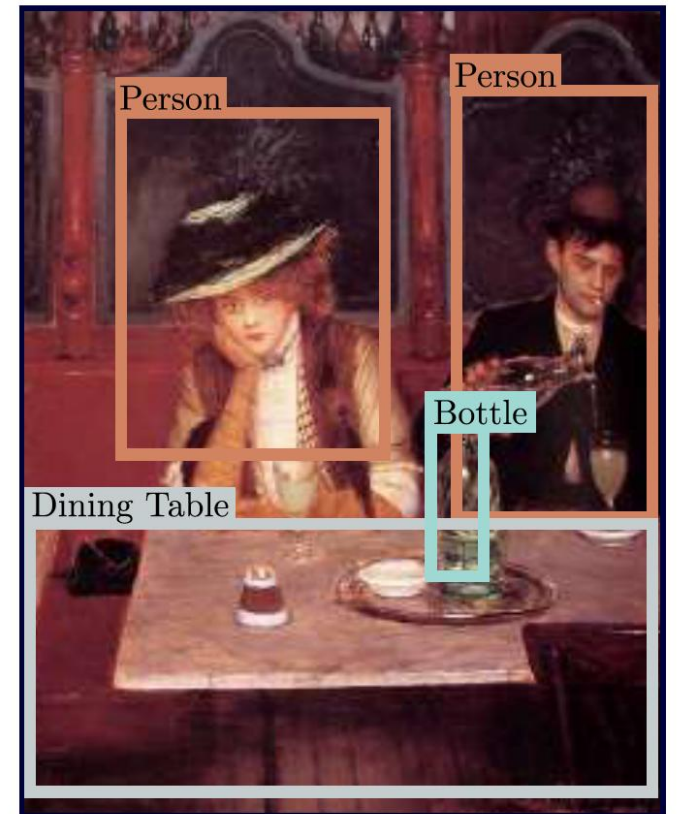
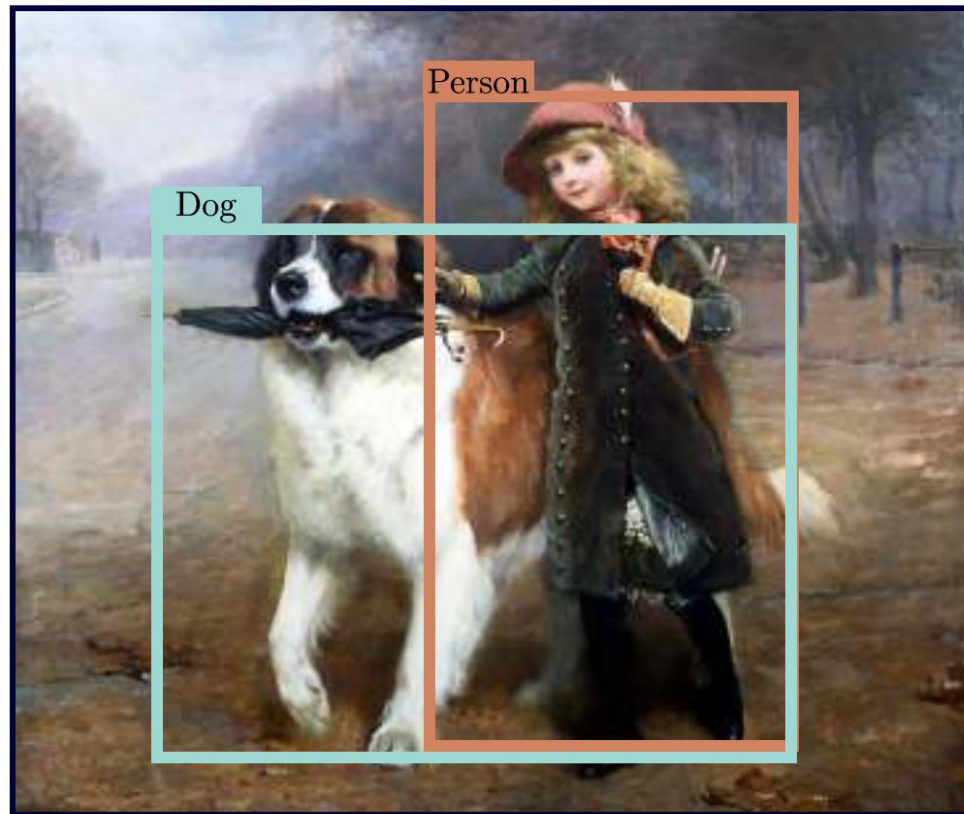
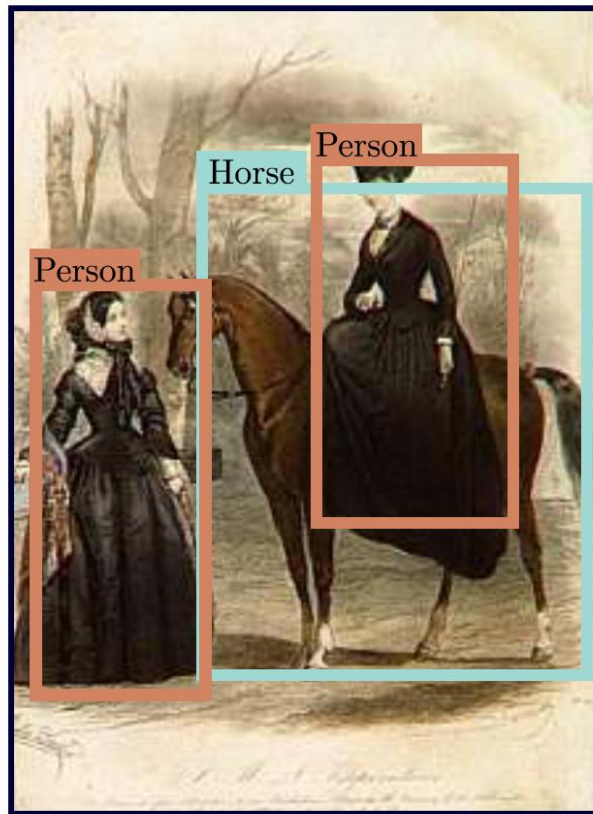
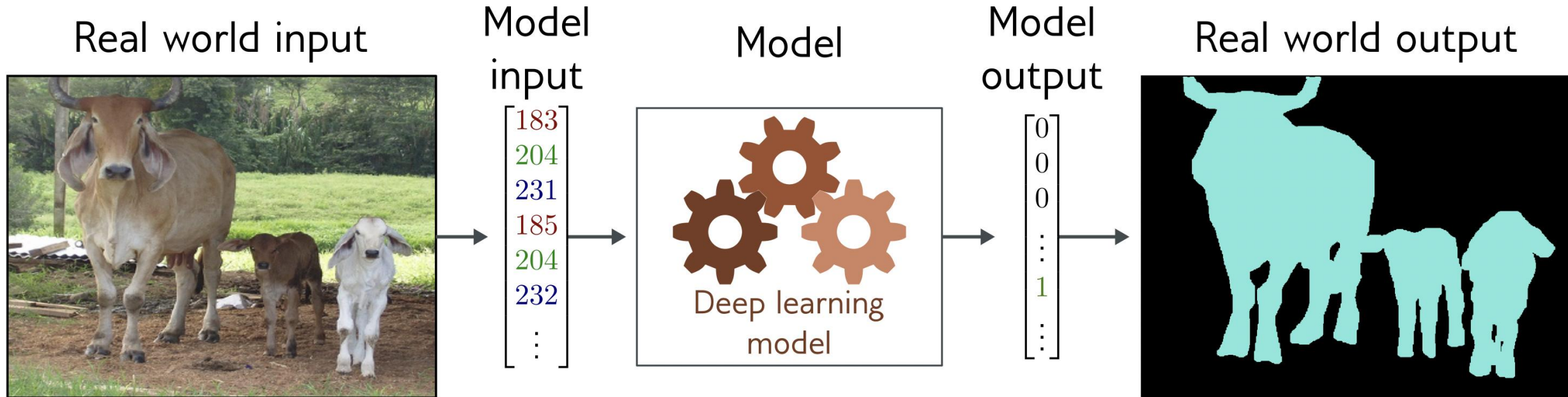
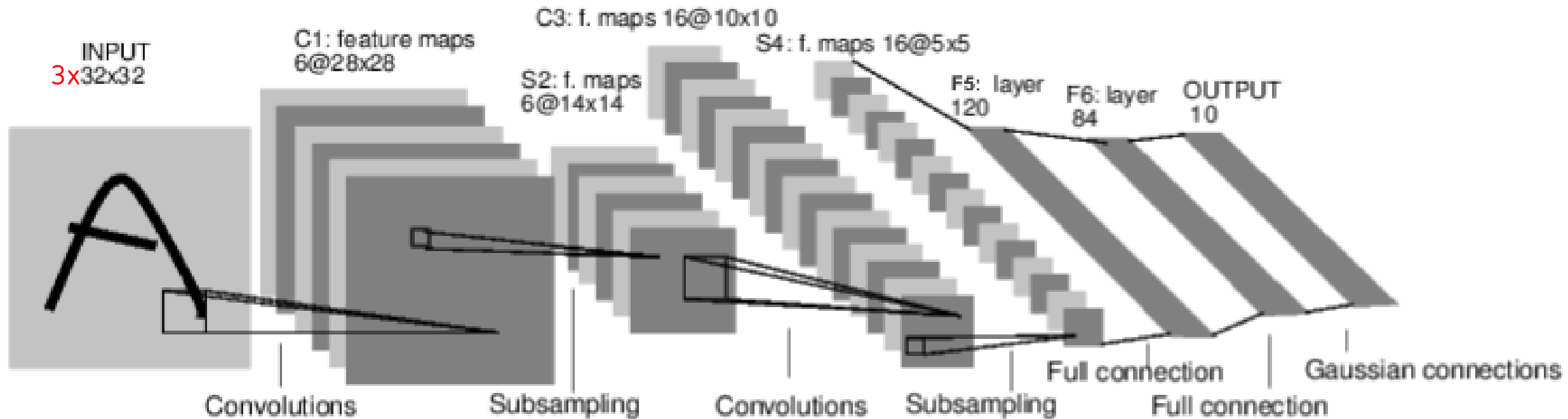


Image segmentation



- Multivariate binary classification problem (many outputs, two discrete classes)
- Convolutional encoder-decoder network

But why don't we just use fully connected networks like we did in lab 2 for classification?



How many “neuron” values in input ? How many “neuron” values in Layer 1 ?

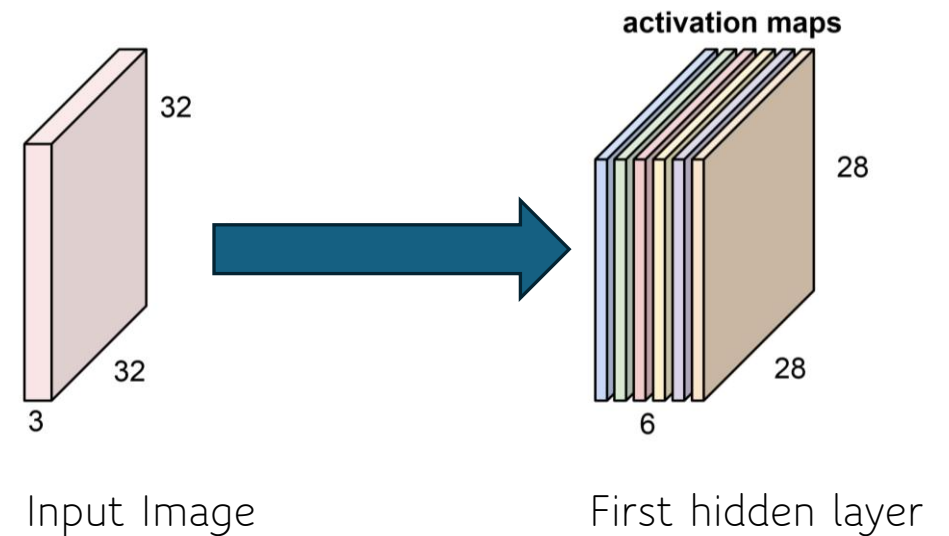
Let us calculate the number of parameters required for a fully connected version of the first layer ...

Input pixel values = $3 \times 32 \times 32 = 3072$

Neurons in first hidden layer = $6 \times 28 \times 28 = 4704$

Total number of parameters = $3072 \times 4704 + 4704$
= 14,455,392

Over **14 million parameters** for a **really small input image** ... very difficult to train and likely to **overfit massively** unless a very large training dataset provided.



How many parameters in the Conv2D Layer ?

Significantly less ...

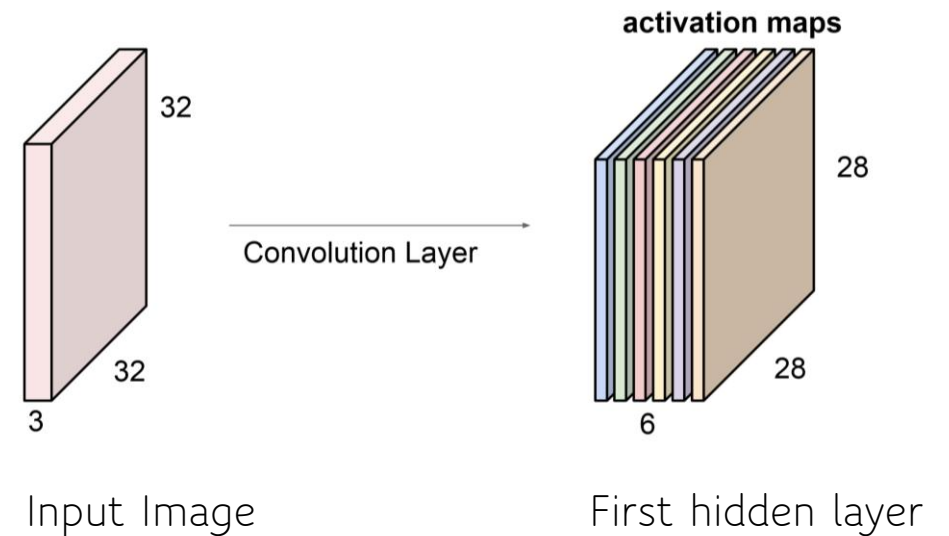
In CIFAR10 classification in Lab 1 you can get the shape of the parameters ...

```
✓ [8] params = list(net.parameters())  
0s print(len(params))  
    print(params[0].size())  
  
10  
    torch.Size([6, 3, 5, 5])
```

So essentially $6 \times 3 \times 5 \times 5 = 450$ weights
(plus another 6 bias terms ...)

Compared to 14 million parameters for the same
transformation of input to output neurons !

This Conv2D layer consists of 6 “kernels” or “filters” with weights of tensor shape 3x5x5 (or 5x5x3 for Tensorflow).



Also CNNs have other desirable properties that fully connected layers do not - **Invariance**

A function $f[x]$ is **invariant** to a transformation $t[]$ if:

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{f}[\mathbf{x}]$$

i.e., the function output is the same even after the transformation is applied.

Invariance example

e.g., Image classification

Image has been translated, but we want our classifier to give the same result



Another desirable property of CNNs: Equivariance

A function $f[x]$ is **equivariant** to a transformation $t[]$ if:

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{t}[\mathbf{f}[\mathbf{x}]]$$

i.e., the output is transformed in the same way as the input

Equivariance example

e.g., Image segmentation

- Image has been translated and we want segmentation to translate with it



How are these convolutional kernels applied to images?

We shift them across the entire image calculating “dot products” between the kernel / filter and the image at each position ... generating another “image” or “activation map”.

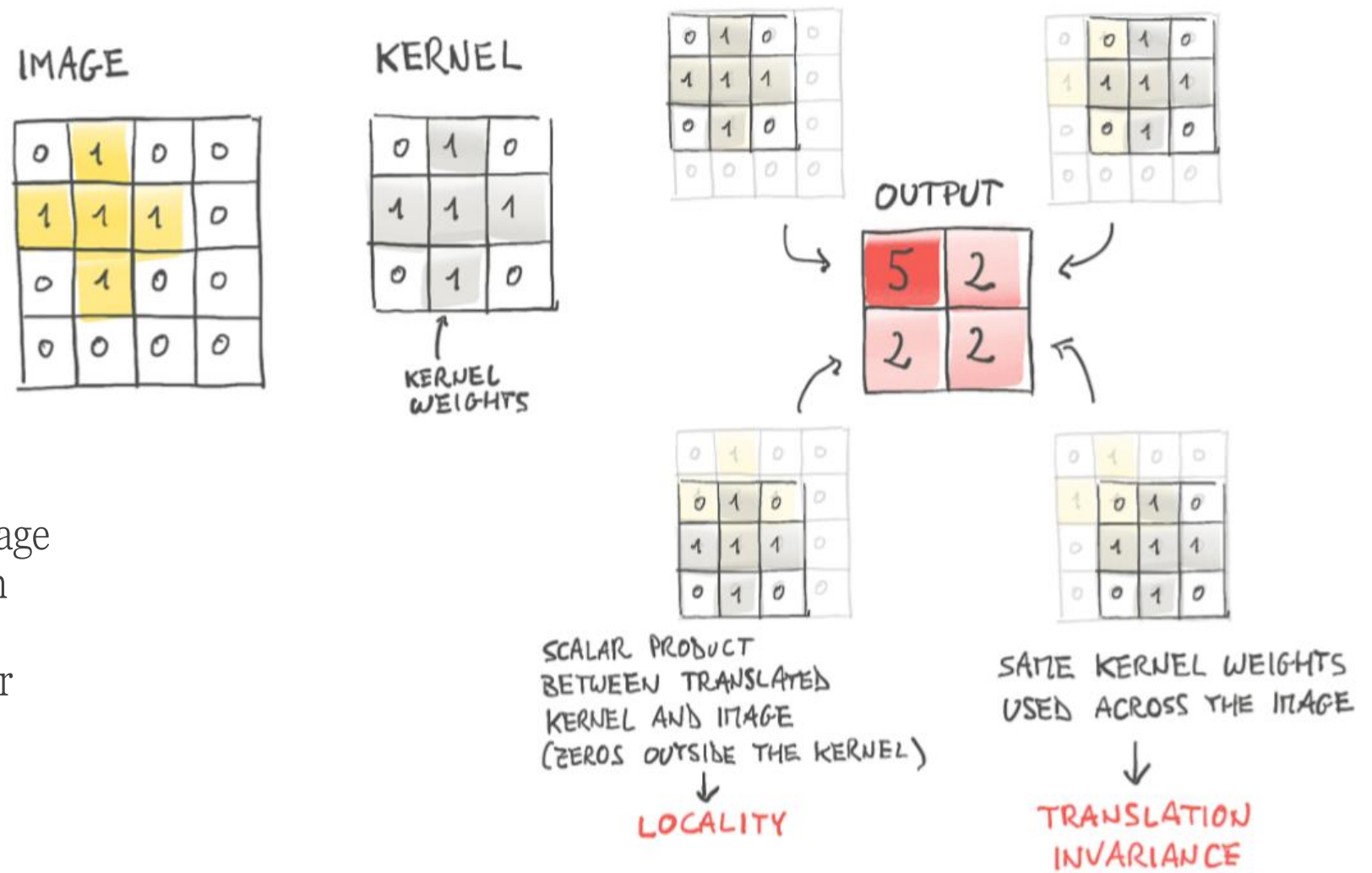
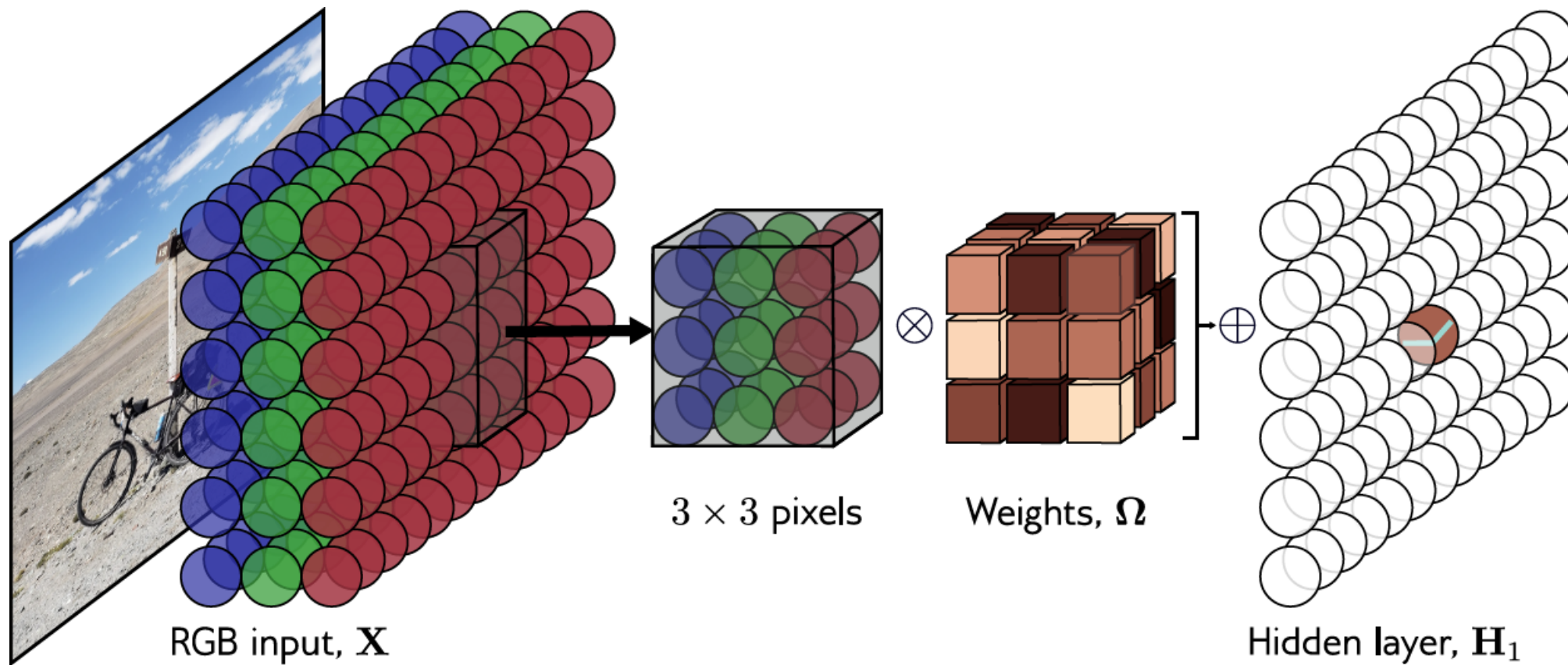


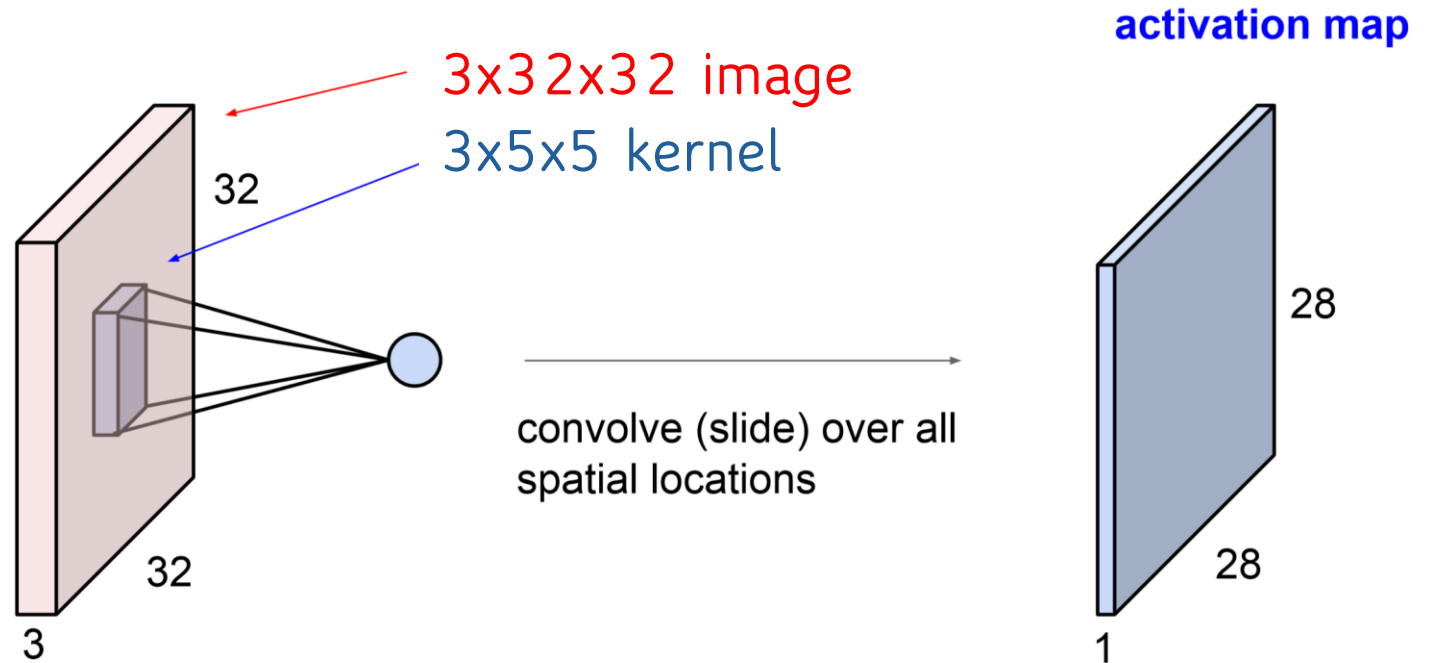
Figure 8.1 Convolution: locality and translation invariance.

3 channels (RGB) for colour images



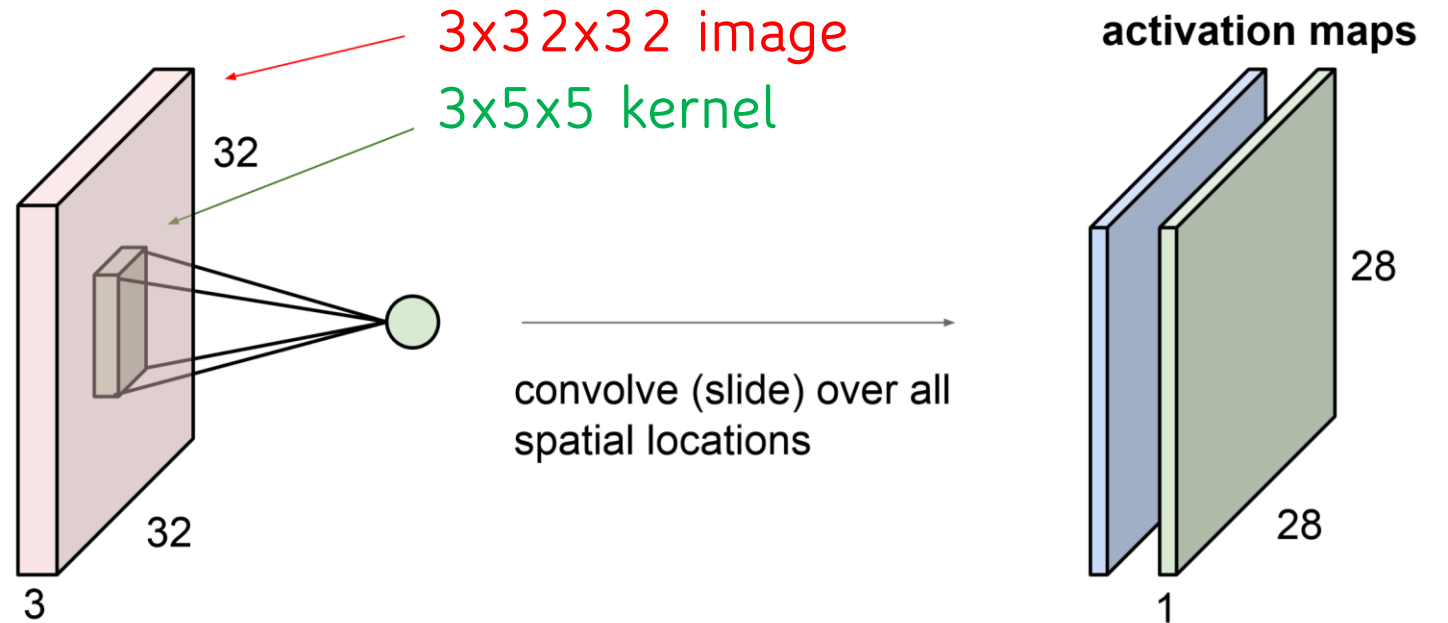
Applying one kernel to a complete image

Why is the output activation map of size $1 \times 28 \times 28$?



Two kernels ...

Each kernel (with different weights) scans the image and produces one output activation map.



All six kernels in the first Conv2d Layer

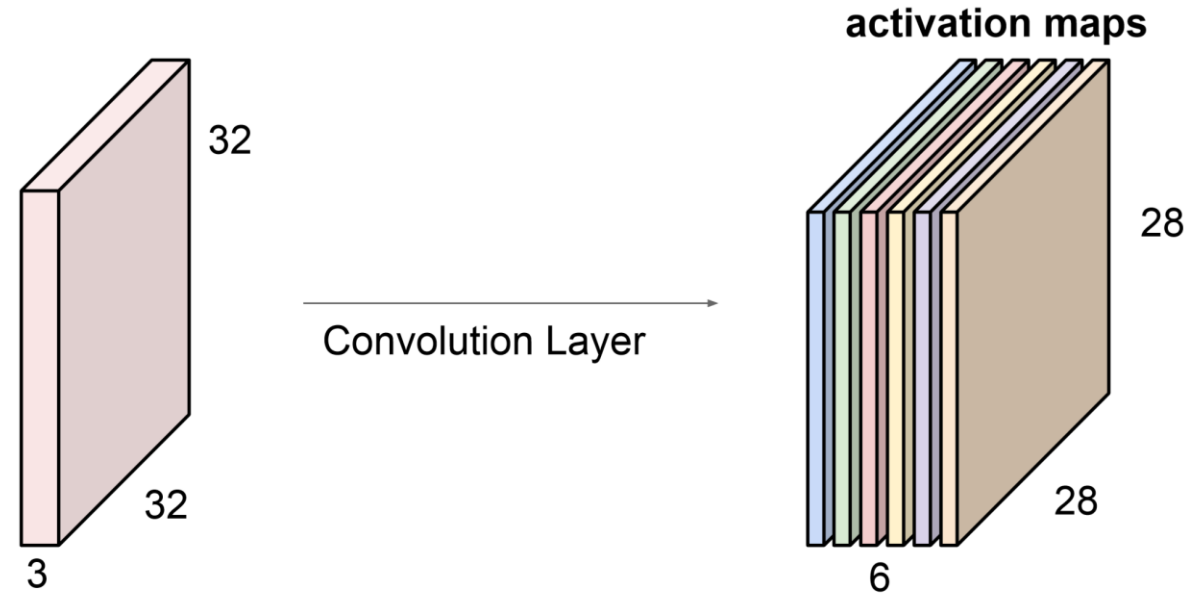
In the PyTorch code:

```
self.conv1 = nn.Conv2d(3, 6, 5)
```

Essentially consists of 3 input channels, and 6 kernels or filters of size 5x5 (creating 6 output channels).

So generates an activation map of size 6x28x28.

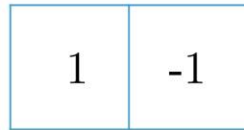
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



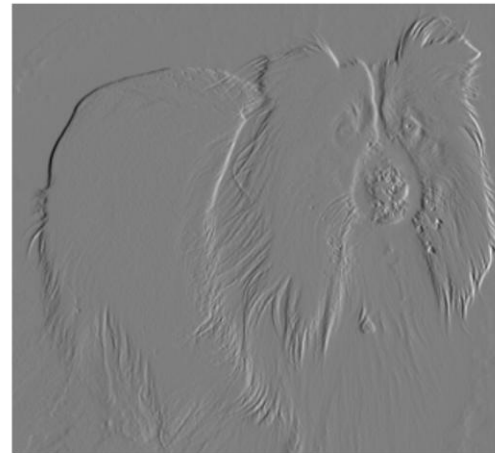
Each of these six kernels may be recognizing different features ... example of edge detection ...



Input

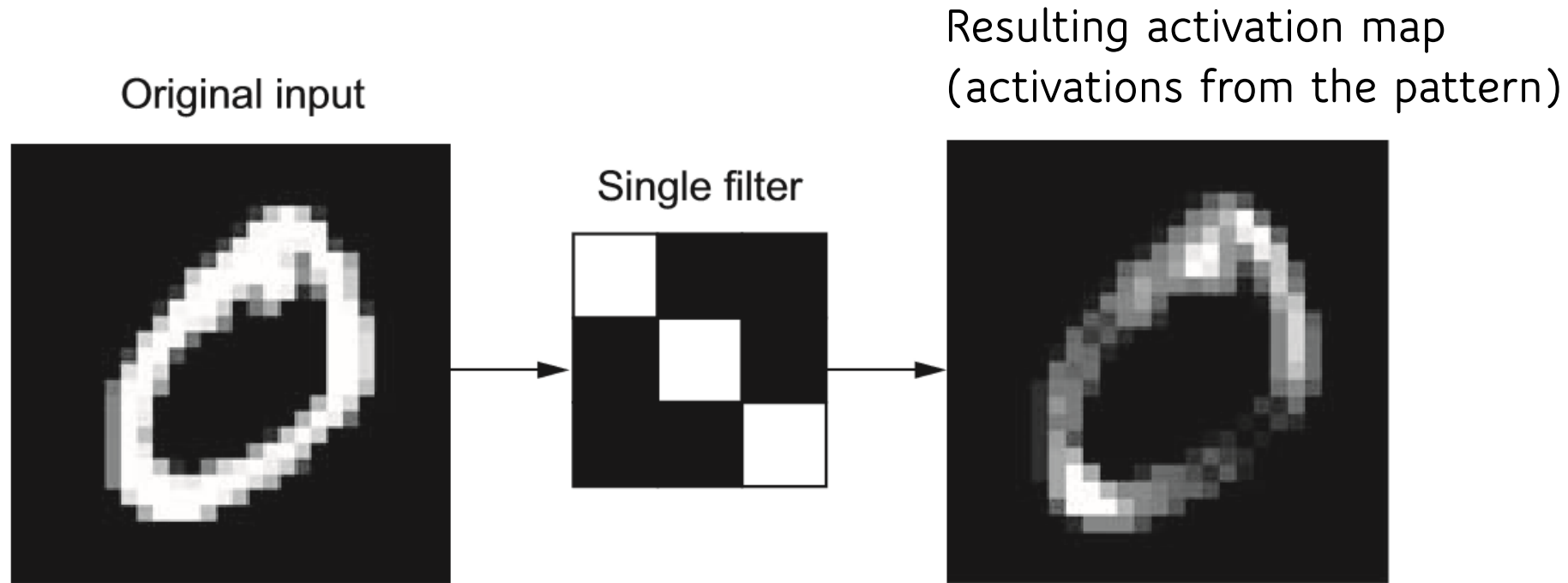


Very simple 1x2 kernel

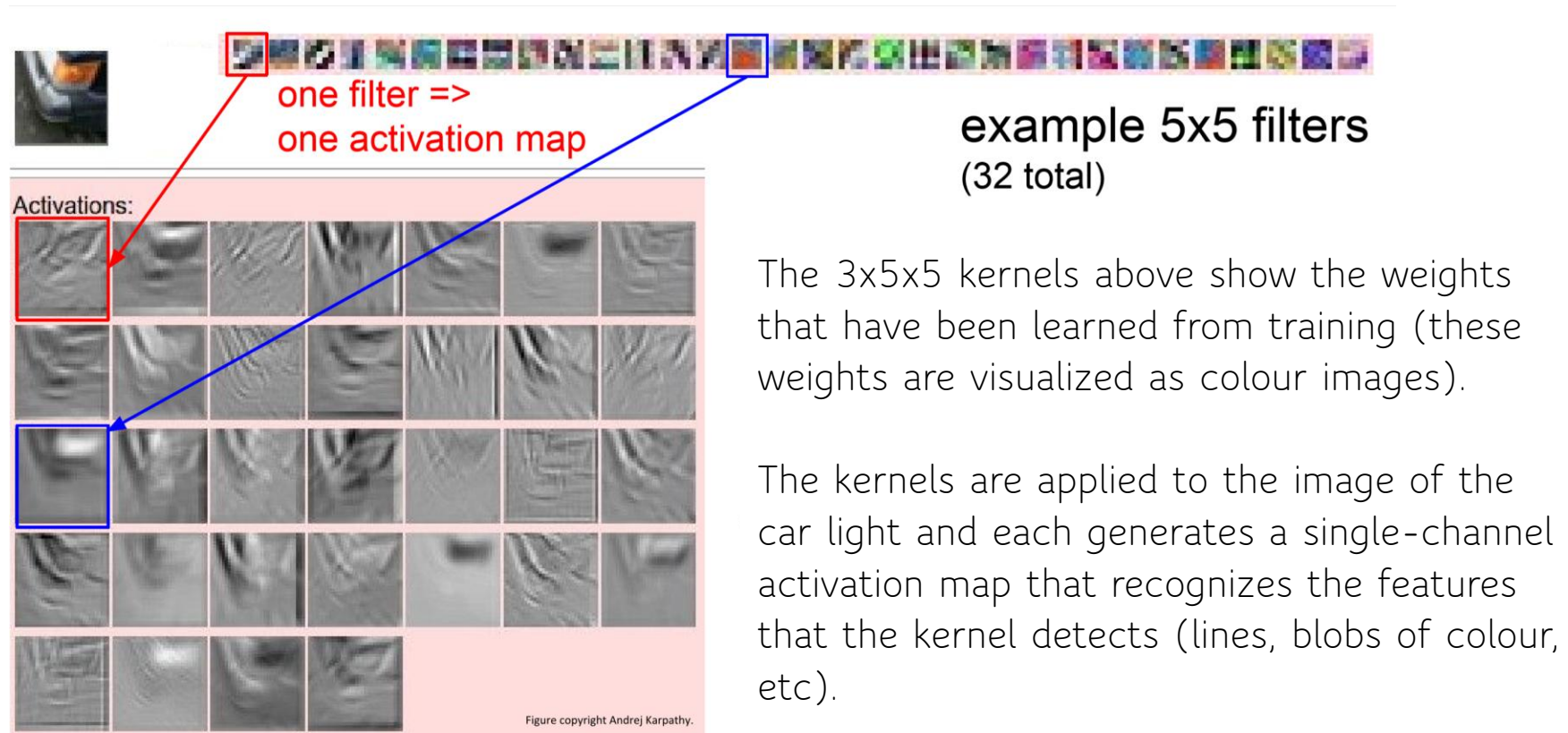


Output

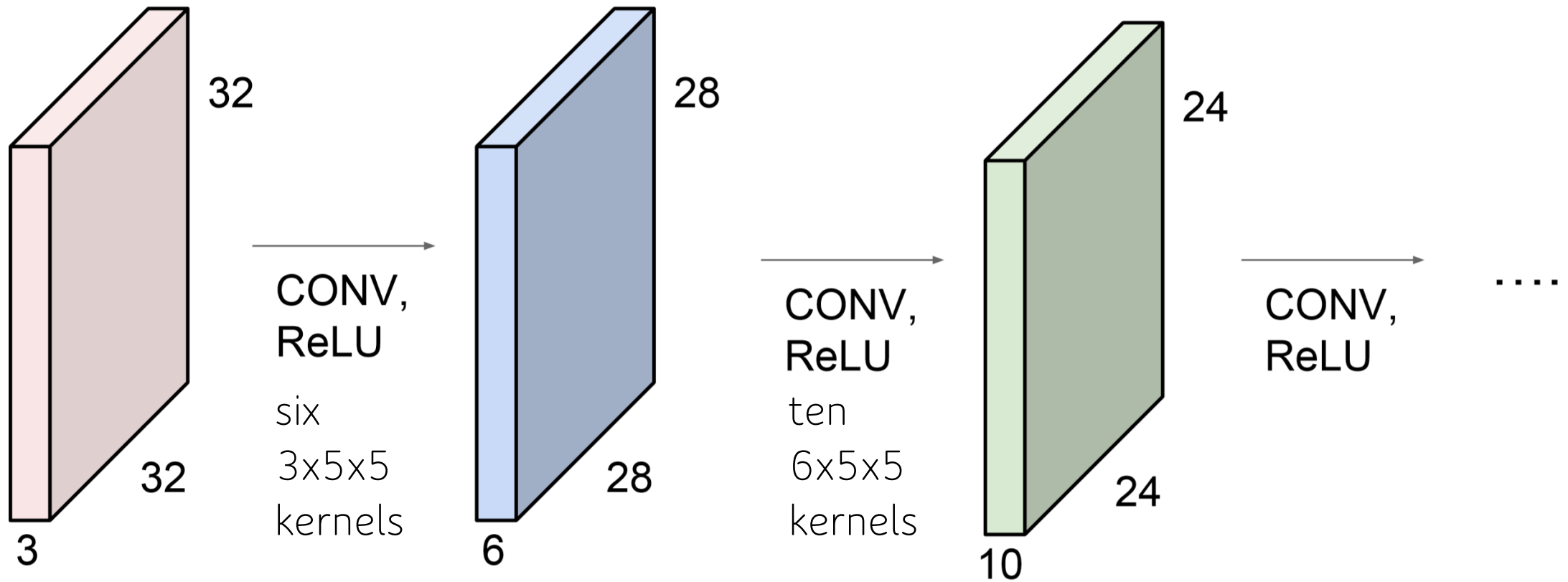
Another might be detecting lines in a particular direction ...



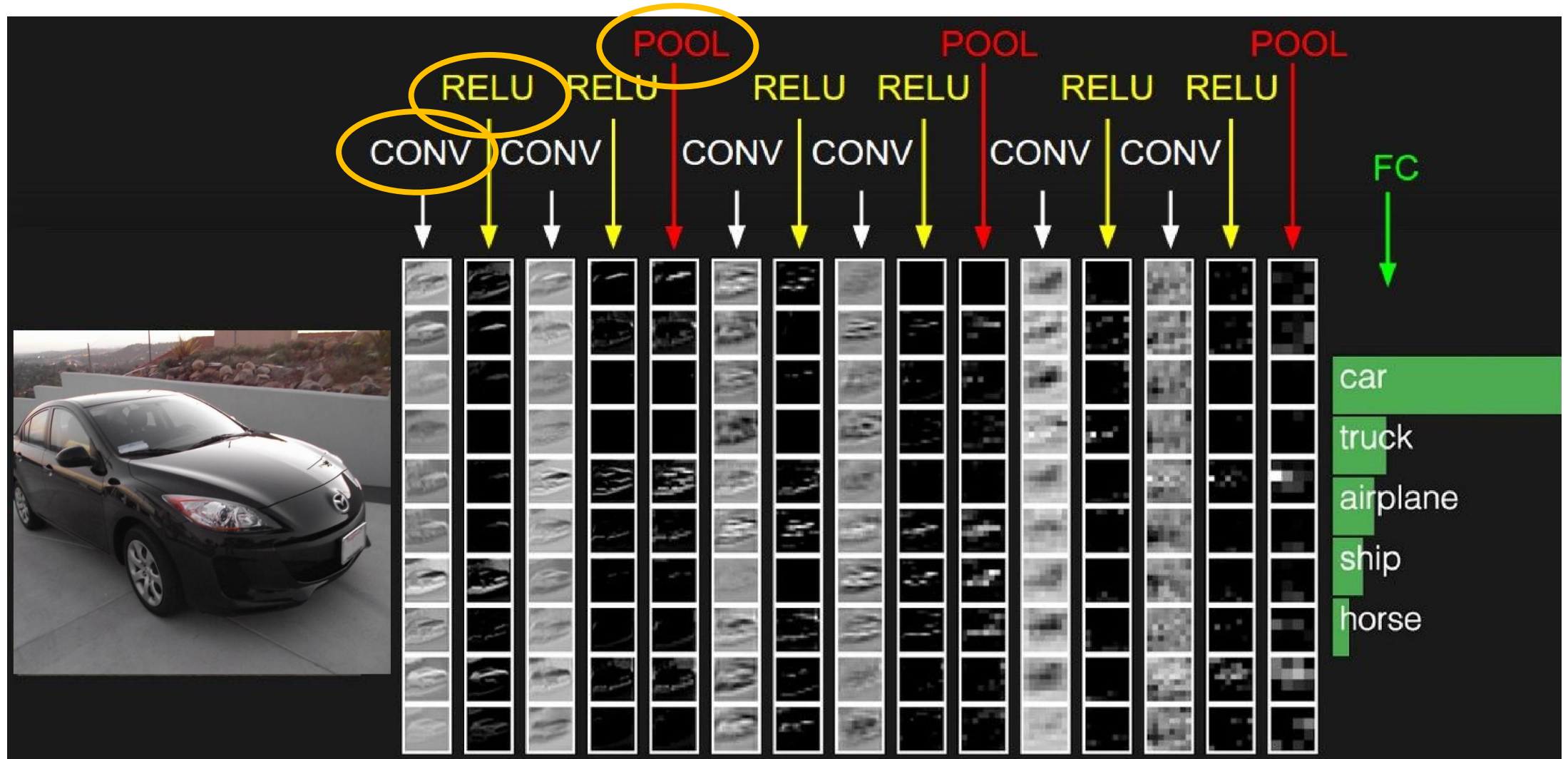
The weights the kernels use are **learned** rather than being designed like these previous examples ...



A ConvNet has multiple convolutional layers with activation functions to recognize hierarchical features ...

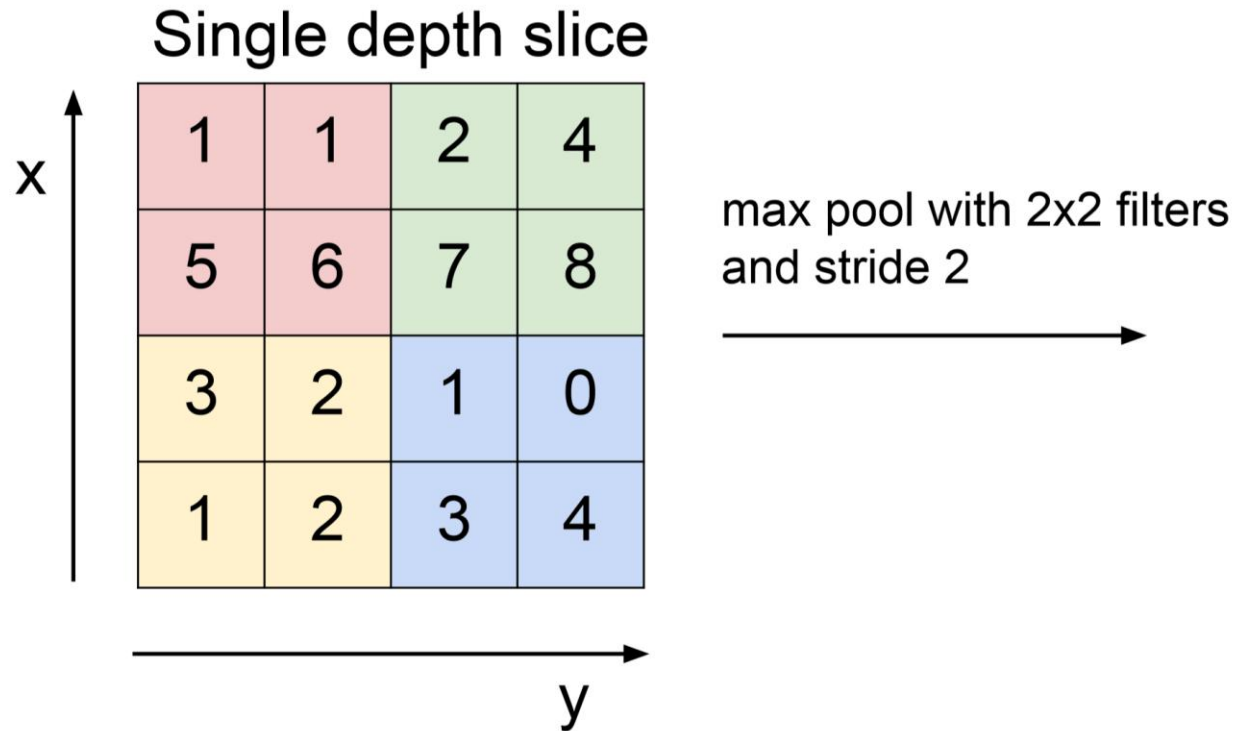


Activation map outputs of deep ConvNet



What are these pooling layers ?

Example of 2x2 Max Pool2d Layer ...



6	8
3	4

In the PyTorch code:

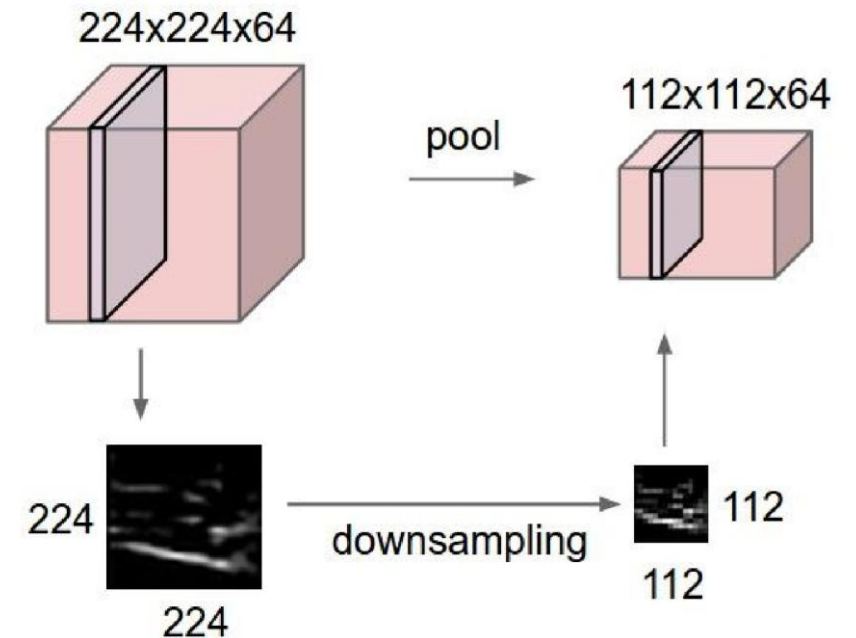
```
self.pool = nn.MaxPool2d(2, 2)
```

Pooling layer

These subsample the input image to reduce the computational load

- makes the representations smaller and more manageable
- Reduces number of parameters to learn
- Emphasizes the maximum activations in a region

Operates over each activation map independently.



Detailed example of all the ConvNet layers ...
 moving the big cross down moves all the activation maps
 (equivariance) and then predicting “cross bottom left”.

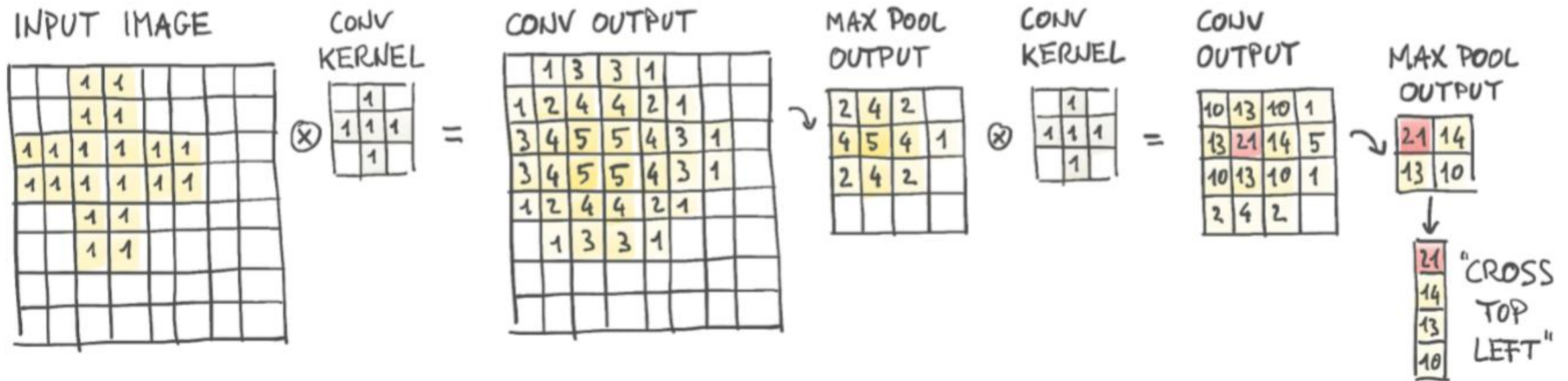
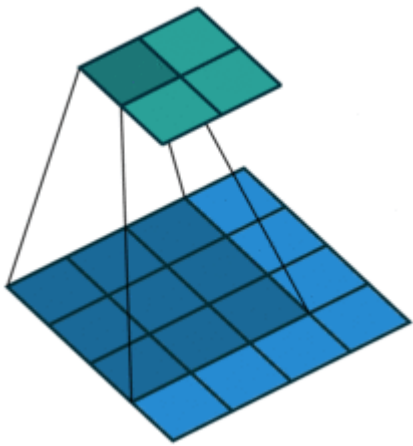


Figure 8.7 Some more convolutions by hand, showing the effect of stacking convolutions and downsampling: a large cross is highlighted using two small cross-shaped kernels and max pooling.

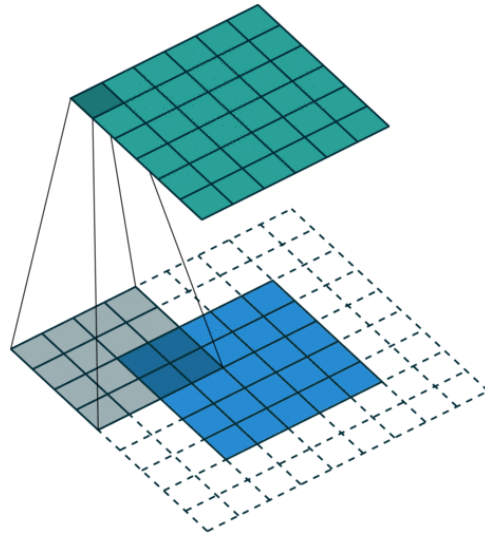
Key aspects of convolutional networks

1. Sparse interactions/connectivity/weights
 - Achieved by making the kernel smaller than the input
2. Parameter sharing so the same “patterns” are detected throughout the image.
3. Equivalence (translational invariance)
4. Sub-sampling (max-pooling) to “concentrate” maximum activations in the image.
5. Fully connected layer to combine the different activations into a classification.

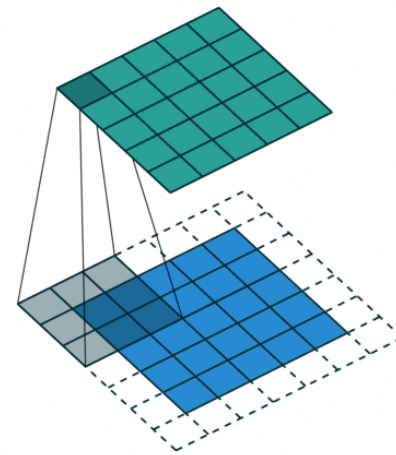
More detailed aspects of CNN layers: **image padding** and stride for Conv Layers



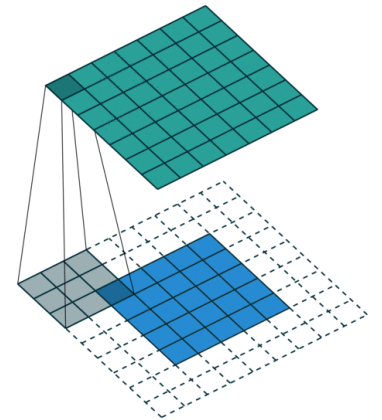
4x4 image
2x2 kernel
Stride = 1
No padding



5x5 image
4x4 kernel
Stride = 1
Padding = 2



5x5 image
3x3 kernel
Stride = 1
“Half padding”
(Output image size
= Input image size)



5x5 image
3x3 kernel
Stride = 1
“Full padding”
(Maximum
padding)

Half padding of images with zeros so they stay the same size

We saw in the previous architecture that the image / activation maps were getting smaller.

Conv' Layers can do "half padding" with zeros so that image / activation maps stay the same size.

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

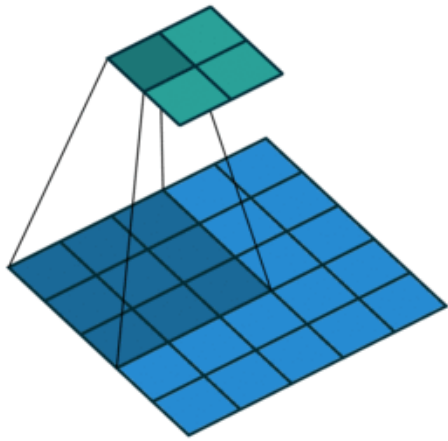
in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

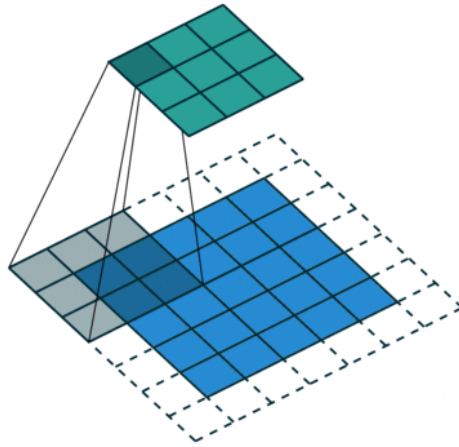
$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

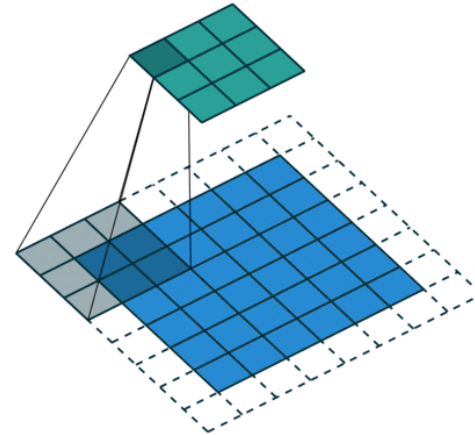
More detailed aspects of CNN layers: image padding and **stride** for Conv Layers (stride can be an alternative to downsample images)



5x5 image
3x3 kernel
Stride = 2
No padding



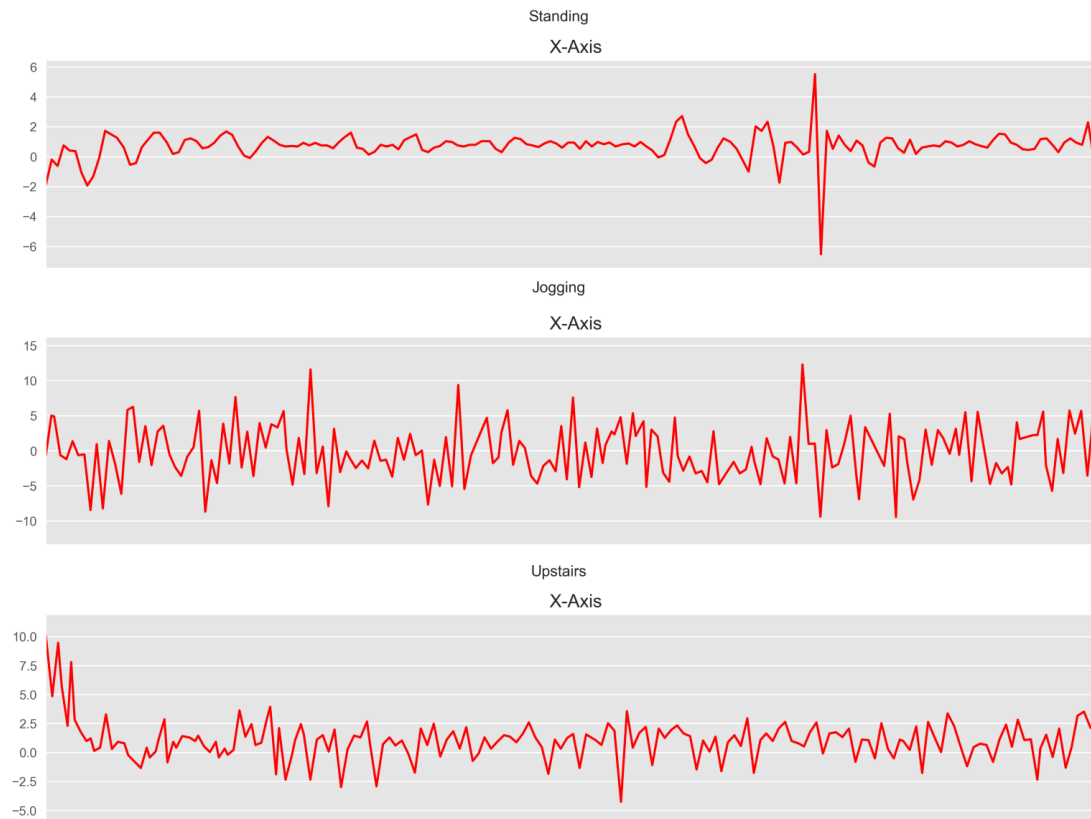
5x5 image
3x3 kernel
Stride = 2
Padding = 1



6x6 image
3x3 kernel
Stride = 2
(with odd size)
Padding = 1

Convolutional networks can also apply to 1D Data

Conv1d Layers ...



For 1D data such as time series and sequence data:

- Audio signals
- Inertial sensing data
- Seismic data
- Protein and DNA sequences
- Etc.

Application of Conv1D kernels on 1D data (showing shared weights)

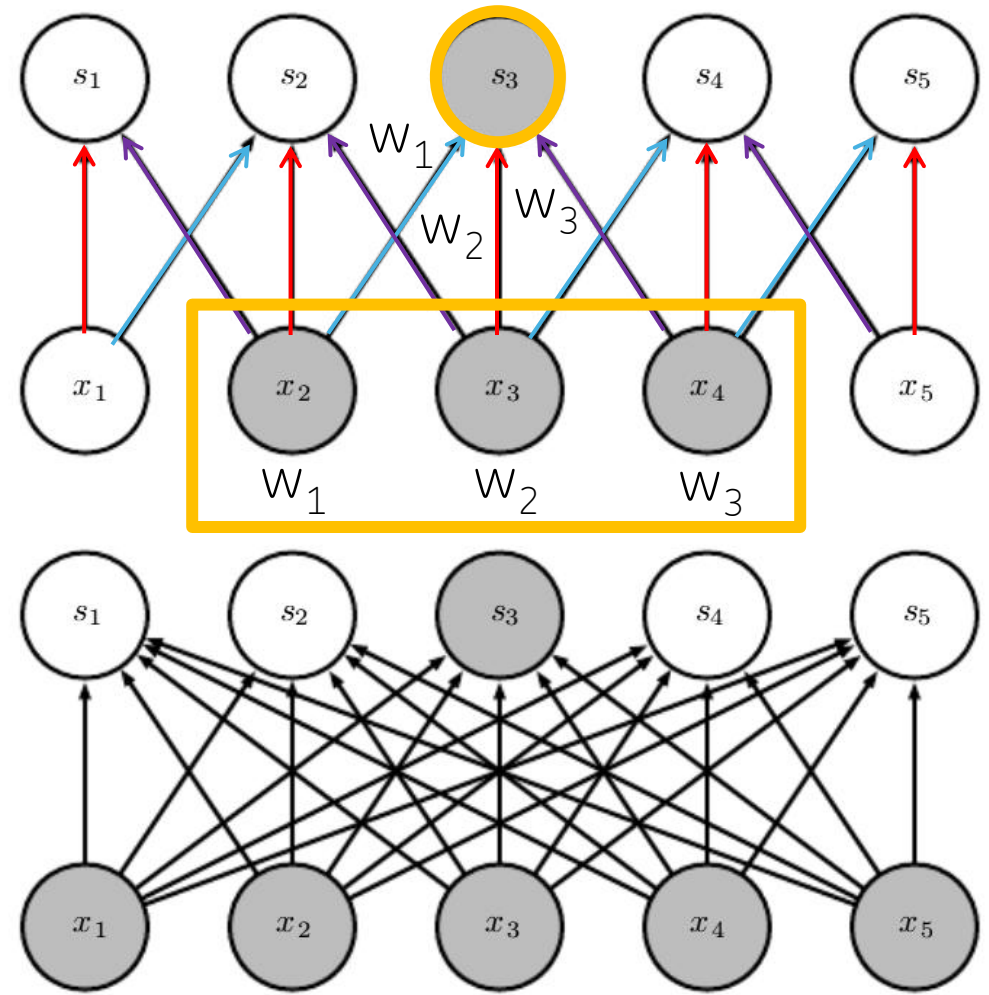
Results in significantly
fewer parameters.

Provides translational
invariance or
equivariance.

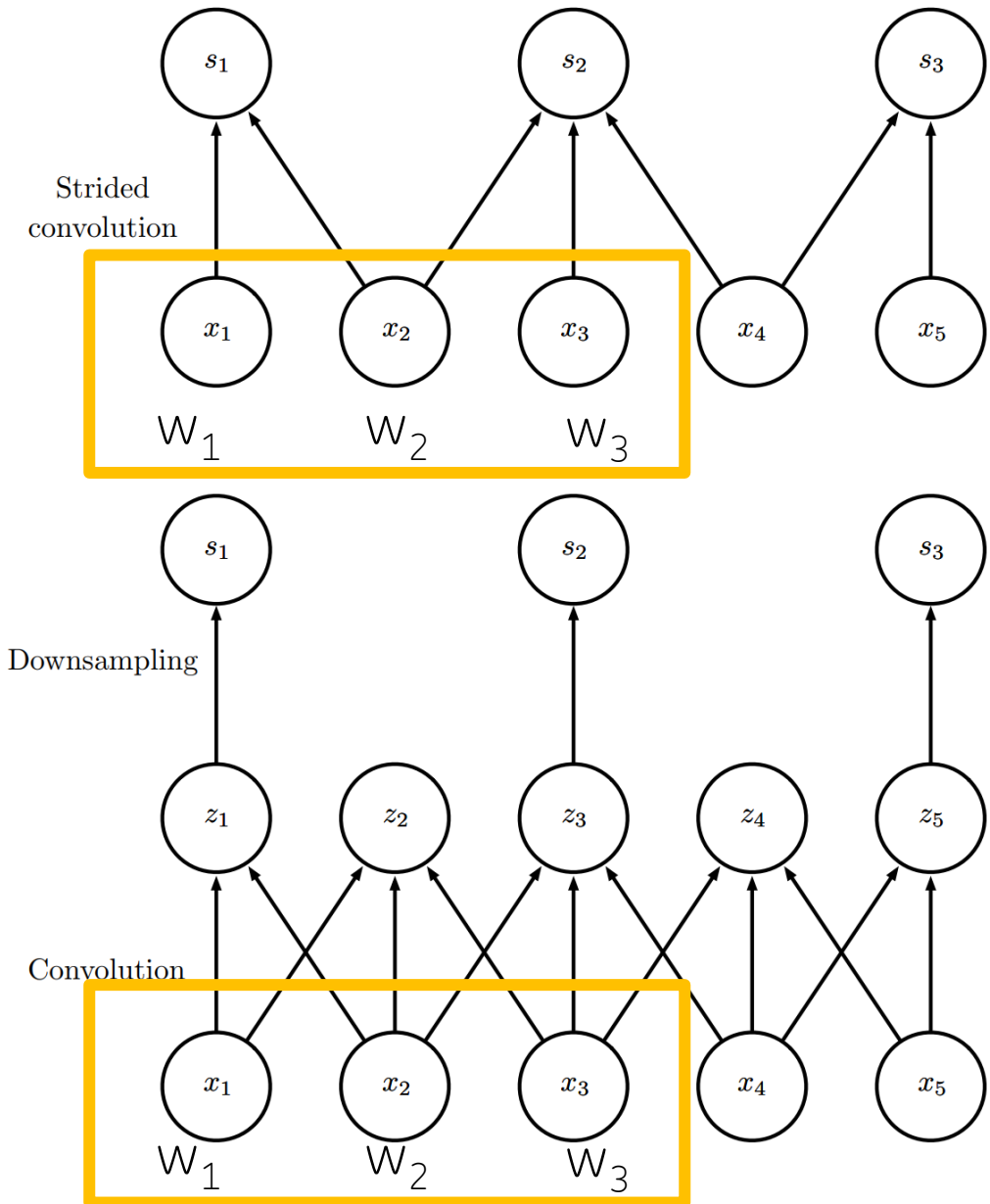
Emphasizes importance
of feature locality.

Conv1D (of length 3)
has only 3 weights
(plus a bias term)

Fully connected has
26 weights with all
different values
(plus 5 bias terms)



Convolutions with “stride”
can be used as a different
sort of downsampling
(instead of max pooling)



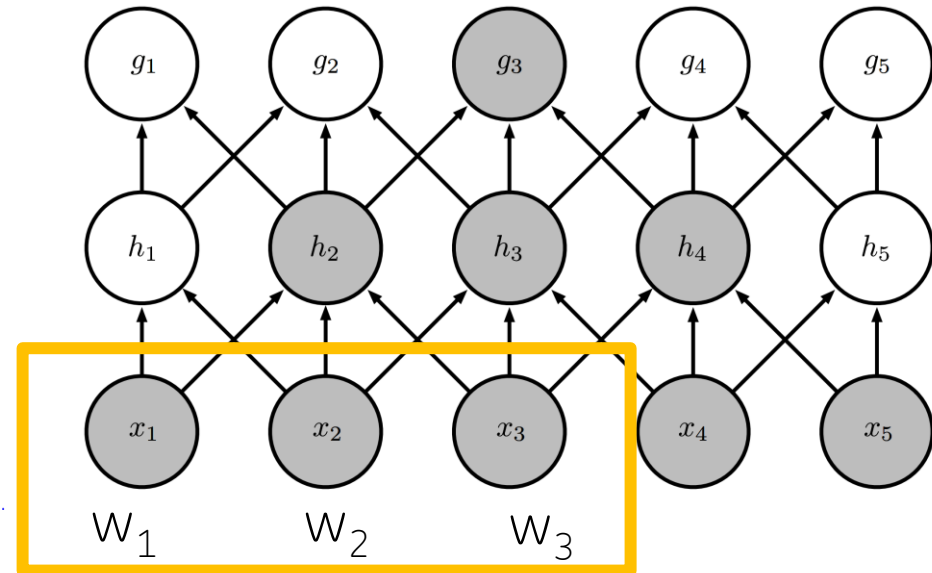
Growing receptive fields with deeper networks

The **receptive field** is the size on the input image that has an effect on a particular neuron.

So g_3 has a receptive field of length 5 here.

This is because we have 2 layers of kernels of length 3 and so the receptive field grows with the number of layers combined.

The trend is for smaller kernels with deeper networks since these require fewer parameters and make maximum use of hierarchical feature combinations.



The “5” neuron in the max pool output has a receptive field of size 4x4 on the original image ... work this out !

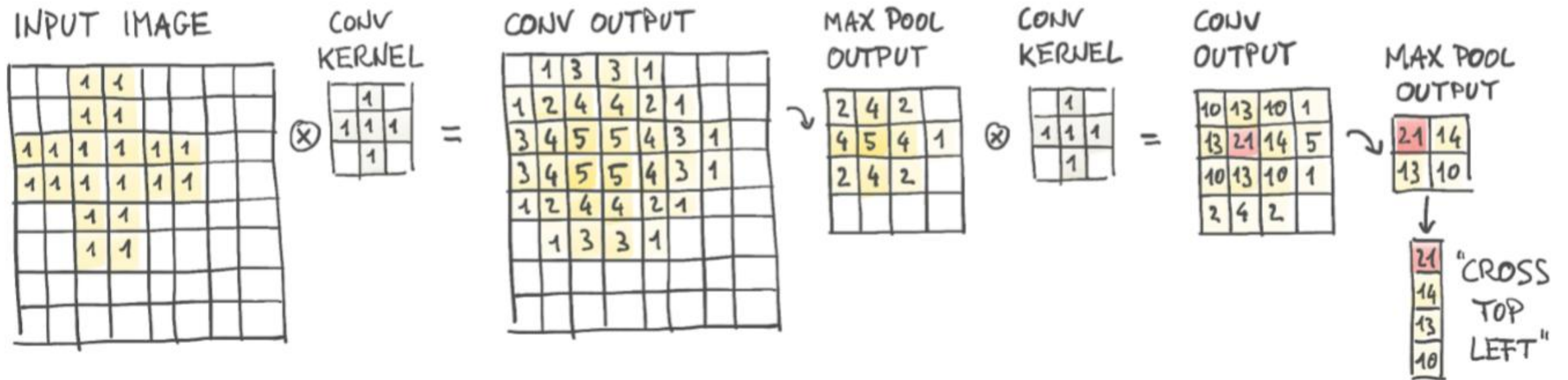


Figure 8.7 Some more convolutions by hand, showing the effect of stacking convolutions and downsampling: a large cross is highlighted using two small cross-shaped kernels and max pooling.

How large is my image after conv / pooling?

Sometimes obvious:

- same size image for 3x3 kernel with padding of 1 and stride of 1.
- 2x2 max-pooling with stride of 2 will half the size of the image

In general filter size F , stride S and padding P ,

$$W_{out} = \frac{(W_{in} - F + 2P)}{S} + 1$$

<https://pytorch.org/docs/stable/nn.html#maxpool2d>

Example of output shape calculation

Input tensor: $3 \times 32 \times 32$ (colour image of size 32x32)

Convolution layer: application of 10 5x5 kernels with stride = 1, padding = 2

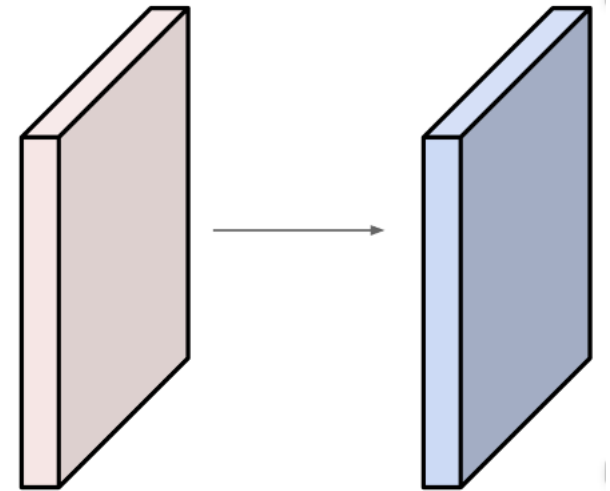
Output volume size:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatial dimension

With 10 kernels, so output tensor is of shape:

$10 \times 32 \times 32$

(This could be done much faster by recognizing this is half padding.)



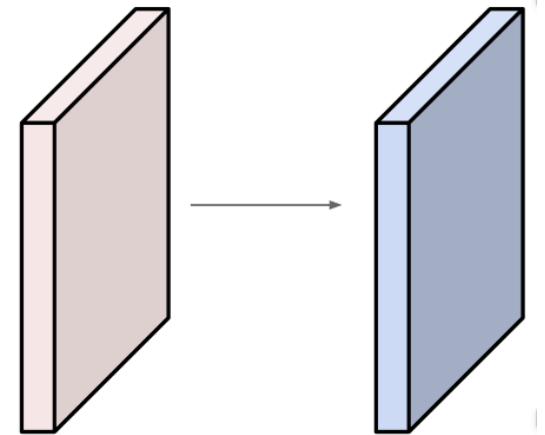
Example of parameter calculation

Input tensor: $3 \times 32 \times 32$ (colour image of size 32×32)

Convolution layer: application of 10 5×5 kernels with stride = 1, padding = 2

Each kernel (or filter) has $3 \times 5 \times 5 + 1$ parameters = 76 parameters.
(The plus one is for the bias term.)

But the layer has 10 kernels so this layer has a total of:
 $10 \times 76 = 760$ parameters.



ConvNet Summary

- ❖ Convolution can be used as the linear operation of a feed forward network dealing with images.
- ❖ Using convolution produces networks with **fewer parameters**, exploiting **locality** and featuring **translation invariance**.
- ❖ Stacking multiple convolutions with their activations one after the other, and using max pooling in between has the effect of applying convolutions to increasingly smaller feature images, thereby effectively accounting for **spatial relationships across larger portions of the input** image as depth increases.
- ❖ Stride can have a similar affect to max-pooling of decreasing the image size as it goes through the network.
- ❖ In an opposite way, Zero padding can be used to make the output image size the same as the input image size if this is a desirable feature to have (for deep residual networks for instance where you want the image size to stay the same).

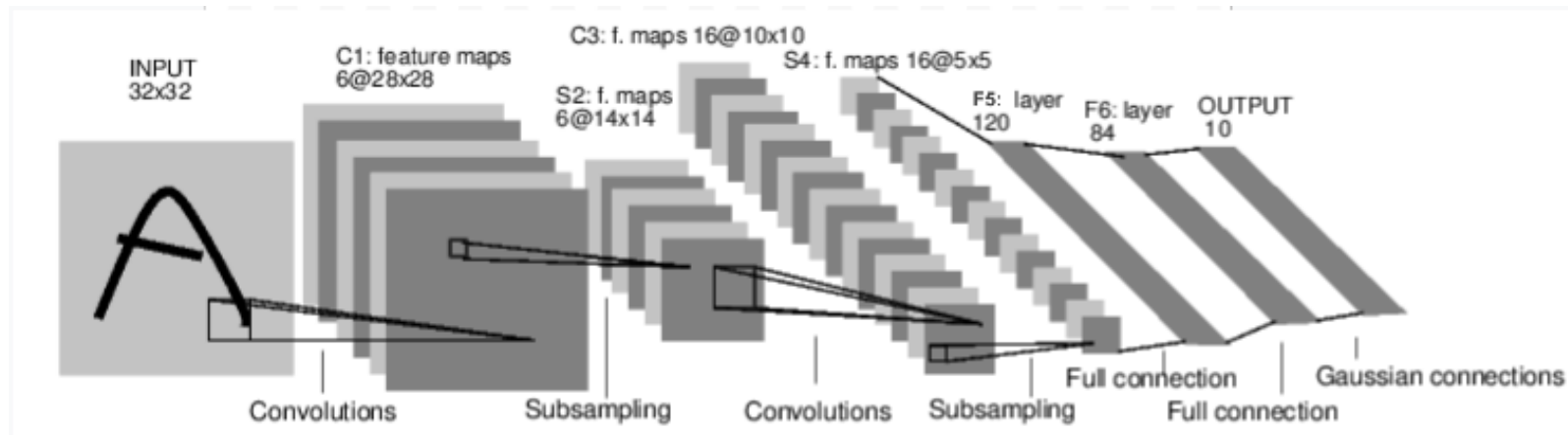
You now should be able to understand the "LeNet-5" Architecture

[LeCun et al., 1998] for MNIST character recognition (28x28 grayscale digit images)

Architecture is [CONV-POOL-CONV-POOL-FC-FC-FC]

Conv filters were 5x5, at stride 1

Subsampling (Pooling) layers were 2x2 max-pooling with stride = 2 (hence halving the image sizes).



2. Define a Convolutional Neural Network

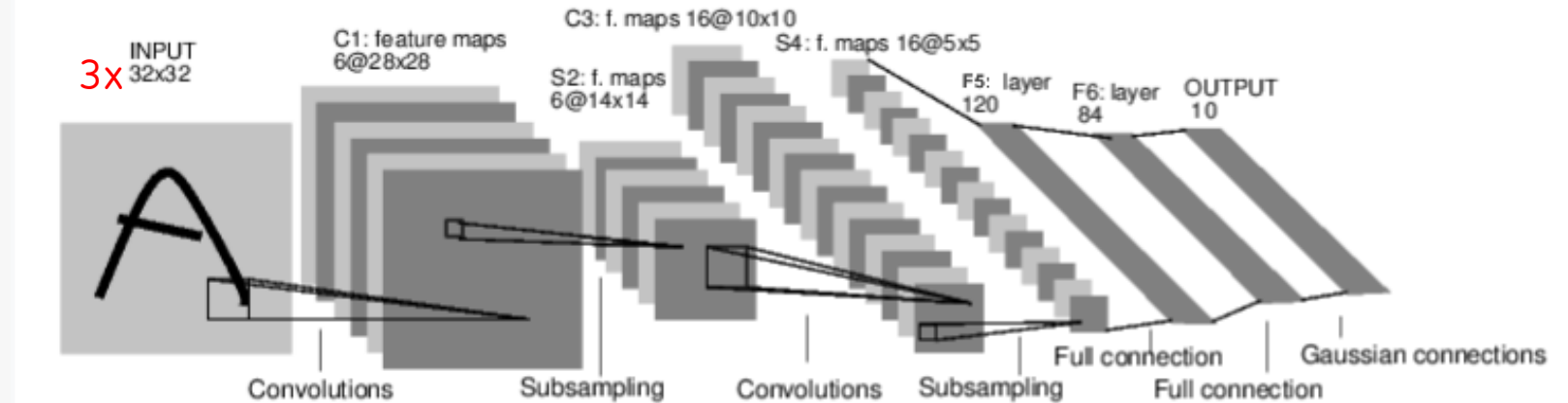
Copy the neural network from the Neural Networks section before and modify it to take 3-channel images (instead of 1-channel images as it was defined).

```
import torch.nn as nn
import torch.nn.functional as F
```

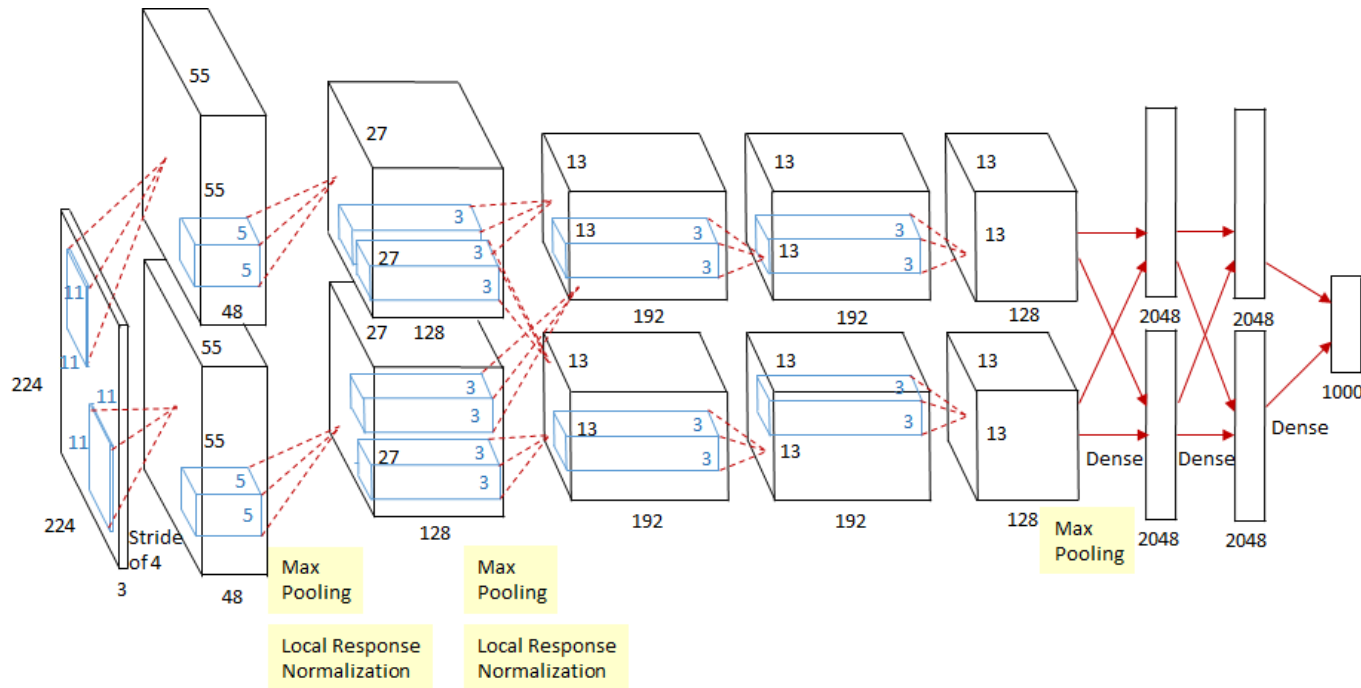
```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```



AlexNet



- Won 2012 ImageNet challenge
- Similar to LeNet-5 but larger and deeper.
- First to stack convolutional layers on top of each other
- Used ReLU's in every layer
- Added two regularisation techniques to reduce overfitting
 1. Dropout
 2. Data augmentation

GoogLeNet

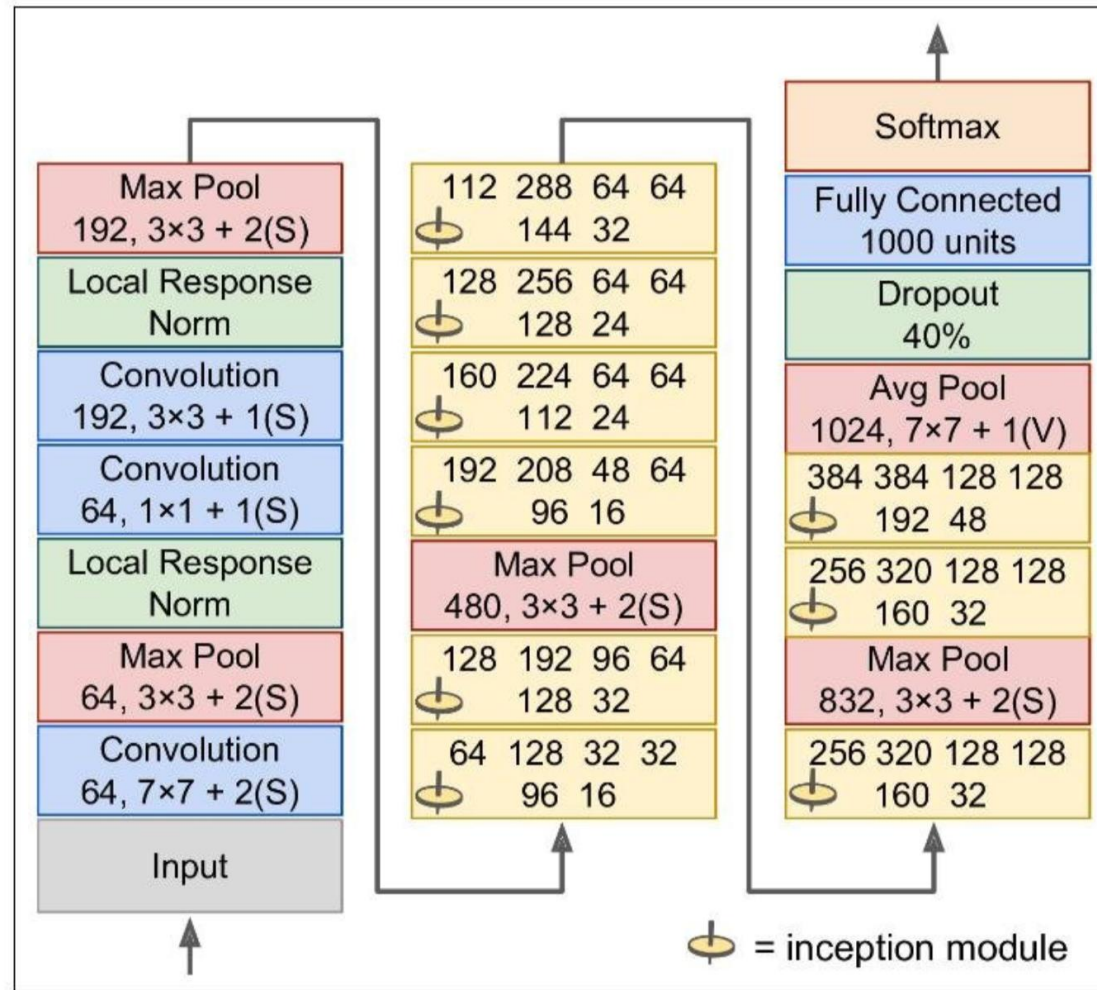


Figure 13-11. GoogLeNet architecture

- Won 2014 ImageNet challenge.
- Has 22 layers ...
- But some are “inception layers”
- Which actually consist of multiple layers themselves ...
- Had to use special incremental training to train such a deep network at this time ...

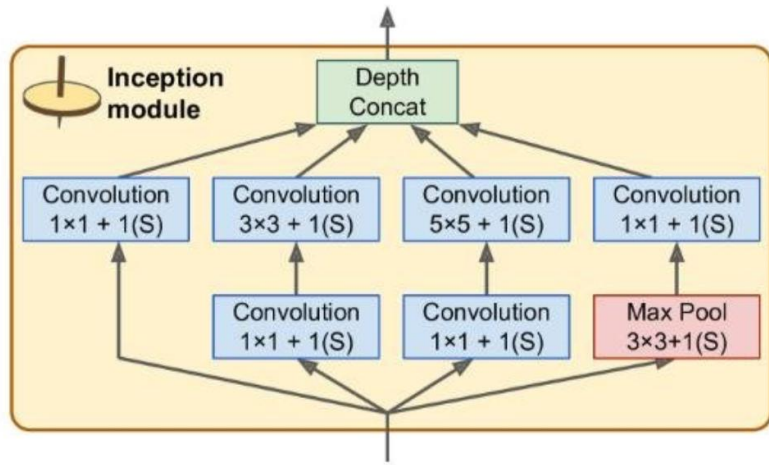
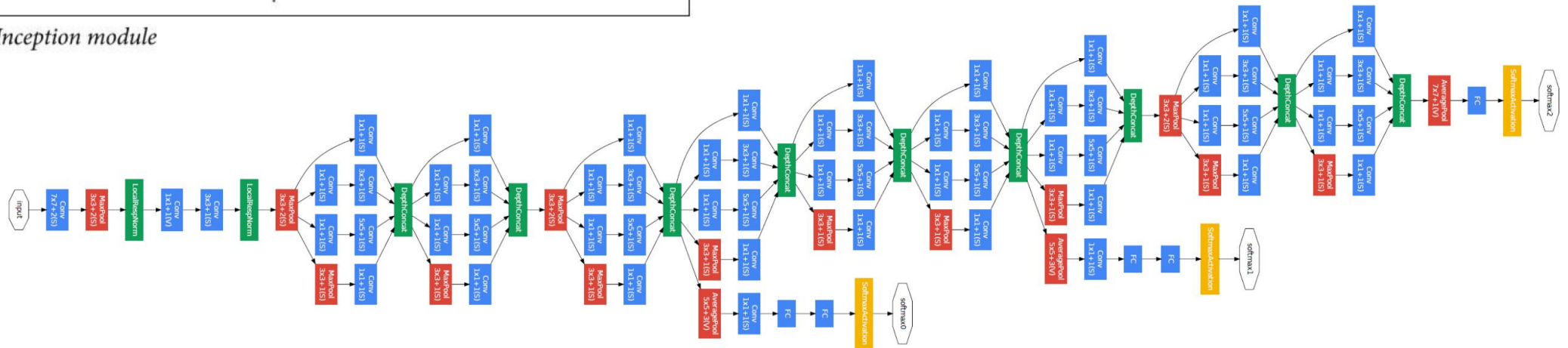


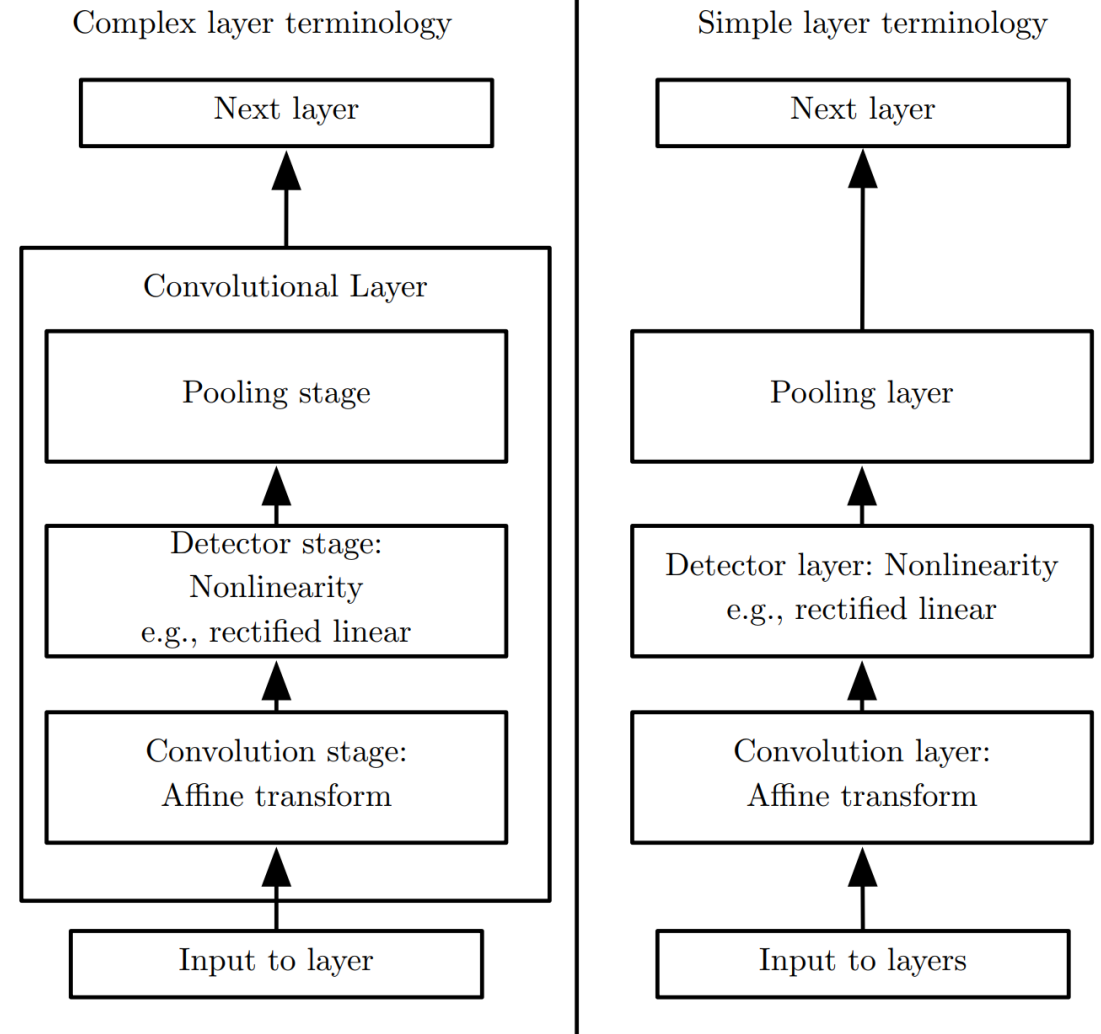
Figure 13-10. Inception module



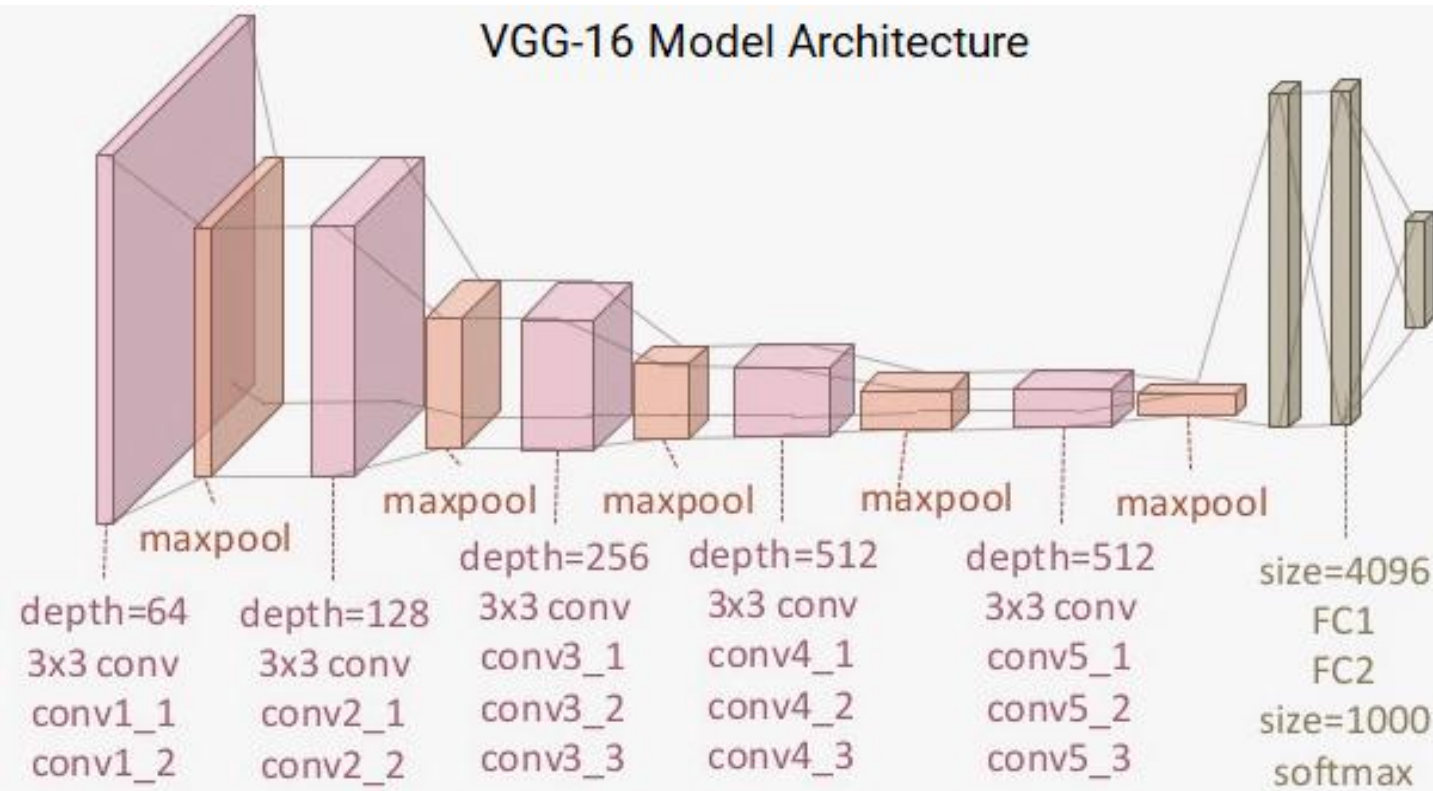
GoogLeNet

INCEPTION MODULES

What constitutes a
“layer” in these very
deep networks
becomes a bit fluid ...



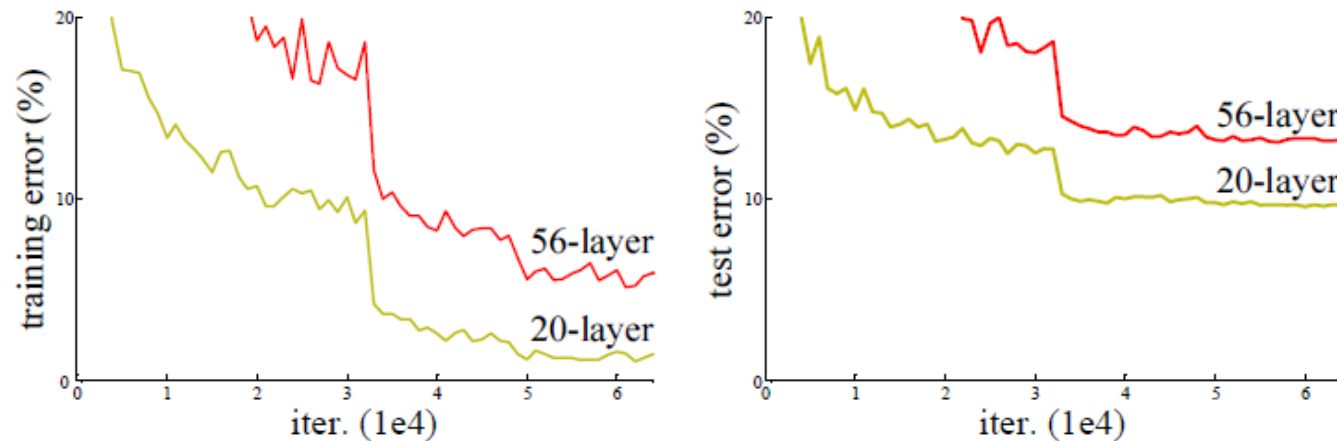
Stacked Higher and Deeper: VGG



- Came runner up in 2014 ImageNet challenge.
- Has 16 layers in this case ...
- Follows trend of deeper networks with larger numbers of smaller kernels (hence deep activation maps)

K. Simonyan, A. Zisserman (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition". *International Conference on Learning Representations*.

What about *very* deep networks?



(results on the CIFAR-10 dataset
<https://www.cs.toronto.edu/~kriz/cifar.html>)

- ❑ Experiments showed that performance starts decreasing beyond a certain depth: this is the degradation problem.
- ❑ More surprisingly, **even the training error** starts increasing, when the depth goes beyond 20!

ResNet – Residual Network with skip connections

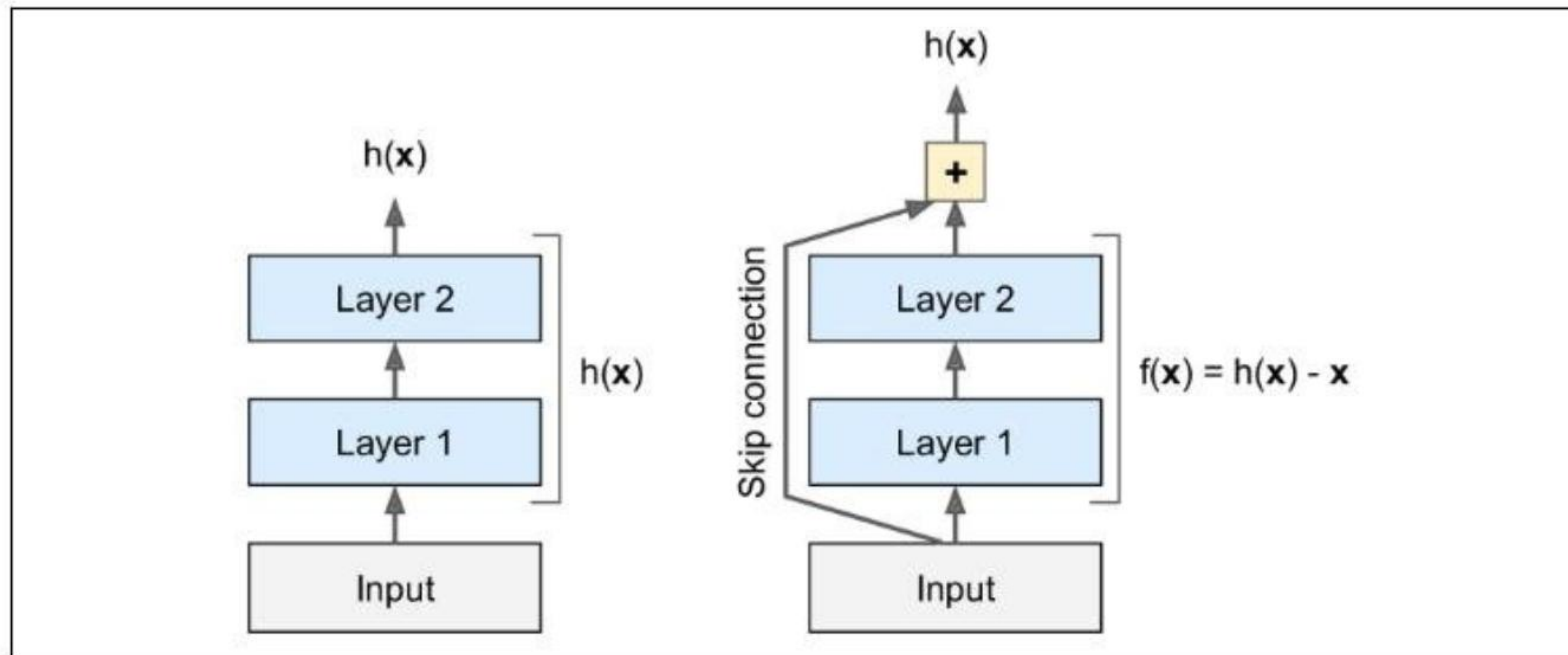


Figure 13-12. Residual learning

Residual units help eliminate issues with *vanishing gradients* ...

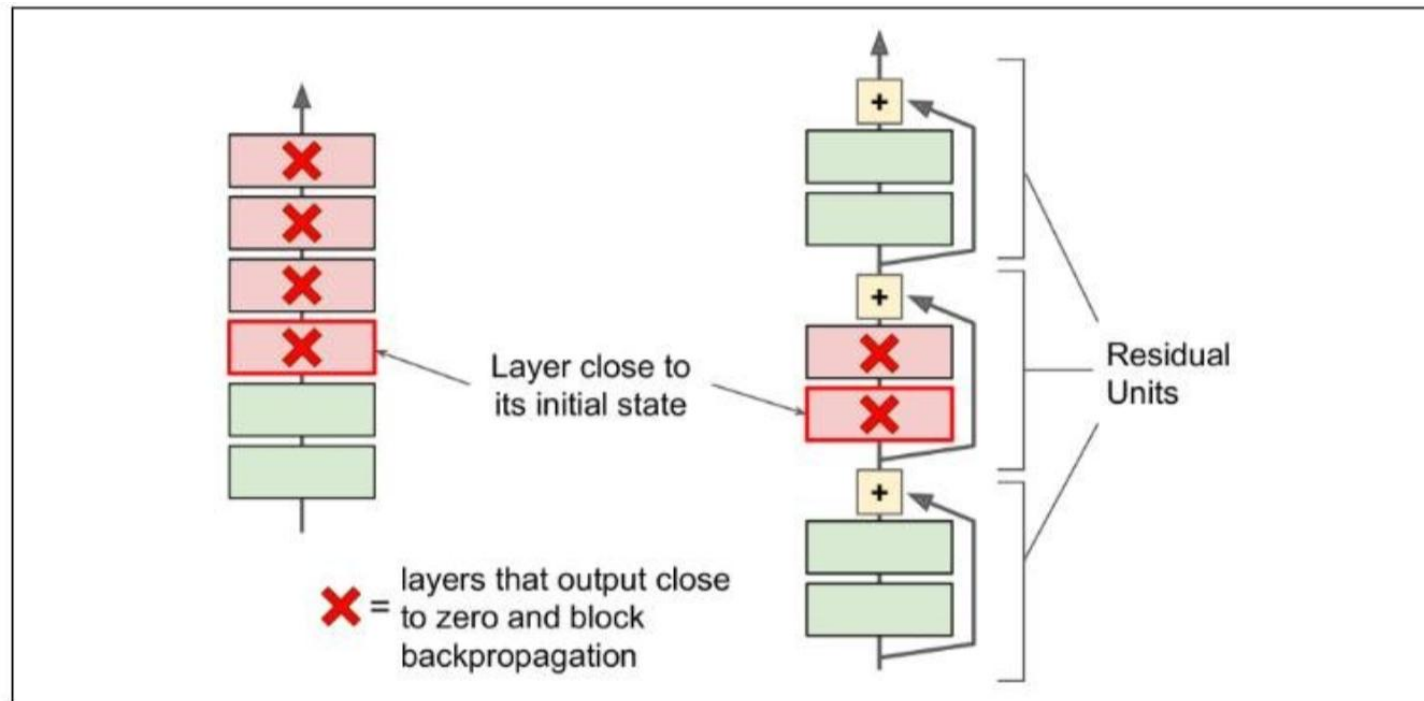


Figure 13-13. Regular deep neural network (left) and deep residual network (right)

ResNet-152

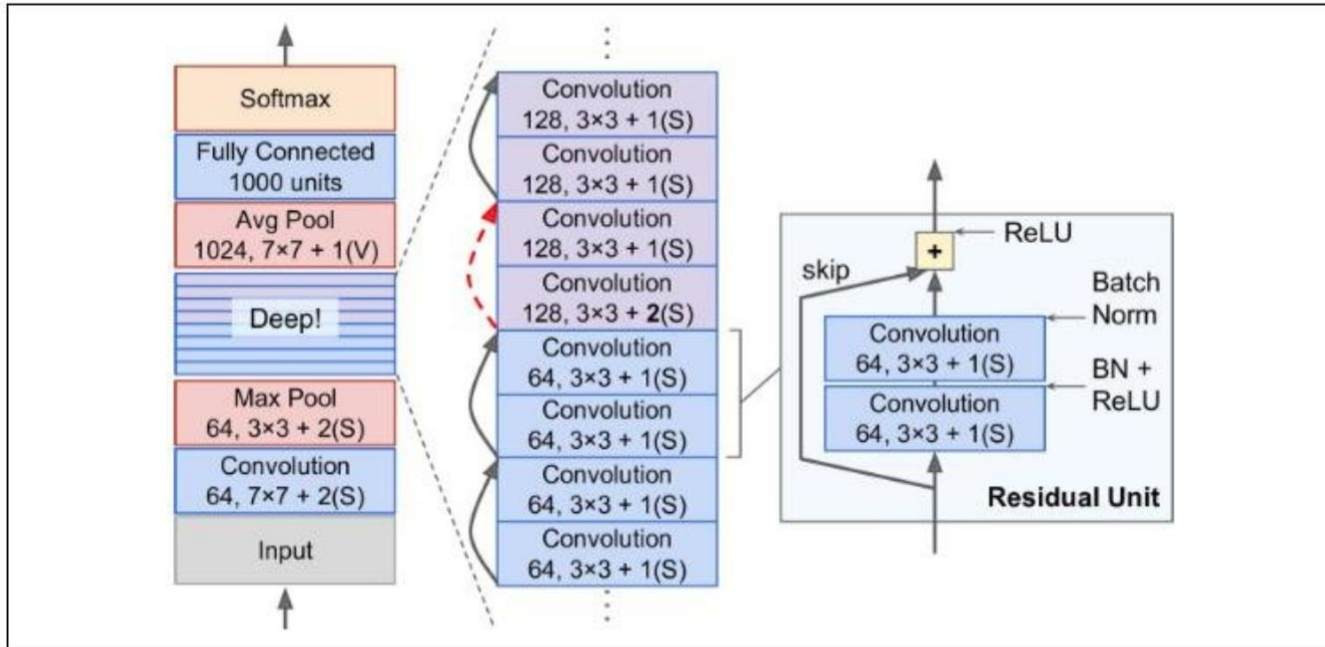


Figure 13-14. ResNet architecture

- Won 2015 ImageNet challenge.
- Has 152 layers but can still be trained due to the residual skip connections !
- Other common versions are ResNet-18, ResNet-50, ResNet-101.

Reading, Theory and Practice

To understand how CNNs work in much more detail:
Chapter 10 of Understanding Deep Learning by Simon Prince.

To get a better formal understanding of the theory of CNNs:
[Chapter 9 Goodfellow et al 2018](#)

Doing Lab 3 (**which the assessment is based upon!**) where you develop a ConvNet (CNN Network) should give you a good idea how this theory relates to implementing a ConvNet in PyTorch.